

# Obere Schranken für die Anzahl der möglichen Schachpositionen

## BACHELORARBEIT

Abgabedatum: 29.09.2023

an der Mathematisch-Naturwissenschaftlich-Technischen  
Fakultät der Universität Augsburg

|                 |                             |
|-----------------|-----------------------------|
| Name:           | Christian Büchs             |
| Matrikelnummer: | 1645356                     |
| Studiengang:    | Wirtschaftsmathematik       |
| Erstgutachter:  | Prof. Dr. Dirk Hachenberger |
| Zweitgutachter: | Prof. Dr. André Uschmajew   |

# Inhaltsverzeichnis

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                               | <b>2</b>  |
| 1.1      | Inhalt der Arbeit . . . . .                     | 2         |
| 1.2      | Was ist eine Schachposition? . . . . .          | 3         |
| <b>2</b> | <b>Shannons Schätzung</b>                       | <b>6</b>  |
| <b>3</b> | <b>Chinchalkars Lösung</b>                      | <b>8</b>  |
| 3.1      | Berechnung der Figurenkombinationen . . . . .   | 8         |
| 3.2      | Platzierung der Figurenkombinationen . . . . .  | 17        |
| <b>4</b> | <b>Tromps Lösung</b>                            | <b>25</b> |
| 4.1      | Fixierte Figuren und Multiplikatoren . . . . .  | 25        |
| 4.2      | Umsetzung der Figurenkombinationen . . . . .    | 29        |
| 4.3      | Berechnung der Bauernverteilung . . . . .       | 30        |
| <b>5</b> | <b>Meine Erweiterung</b>                        | <b>45</b> |
| <b>6</b> | <b>Abschätzung durch ein Konfidenzintervall</b> | <b>54</b> |
| 6.1      | Realisierung der Stichprobe . . . . .           | 54        |
| 6.2      | Mathematischer Hintergrund . . . . .            | 57        |
| 6.3      | Ergebnisse der Stichproben . . . . .            | 59        |
| <b>7</b> | <b>Fazit</b>                                    | <b>61</b> |
| <b>8</b> | <b>Quellenverzeichnis</b>                       | <b>62</b> |
| <b>9</b> | <b>Anhang</b>                                   | <b>63</b> |

# 1 Einleitung

## 1.1 Inhalt der Arbeit

Durch das Internet und die Existenz von Computern wurde unser Leben in den letzten Jahrzehnten in einer Geschwindigkeit revolutioniert, wie noch von keiner anderen Erfindung zuvor. Computer nehmen uns die Navigation mit einer Landkarte durch Google Maps ab, sie machen Briefe durch Apps wie WhatsApp obsolet und in fast allen Lebensbereichen werden sie zur Berechnung und Simulation von Modellen eingesetzt. Deshalb ist es nicht verwunderlich, dass Claude Shannon bereits im Jahr 1950 in seiner Arbeit "XXII. Programming a Computer for Playing Chess" [4] die Frage stellte, ob es möglich sei einen Computer so zu programmieren, dass er Schach perfekt oder zumindest auf dem Level eines guten Schachspielers spielt.

In jener Arbeit verwendete Shannon die beiden Zahlen  $10^{120}$  und  $10^{43}$  als Approximationen für die Anzahl der möglichen Schachspiele und die Anzahl der möglichen Schachpositionen, um zu begründen, dass es keinen perfekt Schach spielenden Computer geben kann, der in einer angemessenen Zeit einen Zug durchführt. Aus diesem Beweis erwuchsen die beiden Fragen: "Was ist die kleinste obere Schranke, die bisher für die Anzahl der möglichen Schachpositionen gefunden wurde?" und "Was ist eine gute Schätzung der Anzahl der möglichen Schachpositionen?"

In dieser Bachelorarbeit beschäftigen wir uns mit diesen beiden Fragen. Wir werden die erste Frage beantworten, indem wir die verschiedenen Versuche von Chinchalkar, Tromp und mir betrachten, eine möglichst kleine obere Schranke zu finden und das jeweilige Vorgehen beweisen. Für die Beantwortung der zweiten Frage werden wir zu Beginn Shannons Schätzung betrachten und als Abschluss Tromps Berechnung eines Konfidenzintervalls studieren.

Diese Arbeit setzt voraus, dass die Schachregeln bekannt sind. Eine Schachanleitung ist unter <https://www.brettspielnetz.de/spielregeln/schach.php> zu finden. Darüber hinaus werden für das sechste Kapitel Kenntnisse der Vorlesung Stochastik 1 vorausgesetzt.

## 1.2 Was ist eine Schachposition?

Bevor wir uns oberen Schranken und Konfidenzintervallen widmen können, müssen wir definieren, was eine Schachposition überhaupt ist. Eine Schachposition soll eine Spielsituation beschreiben. Im Schach können sich zwei Spielsituationen voneinander in folgenden Aspekten unterscheiden:

- 1) **Figurenverteilung**  $F$ : Enthält für jedes der 64 Felder die Information, ob eine bzw. welche Figur darauf steht.
- 2) **Zugrecht**  $Z$ : Es gibt zwei Möglichkeiten für  $Z$ :
  - Weiß ist am Zug.
  - Schwarz ist am Zug.
- 3) **Rochade-Situation**  $R$ : Für jeden Spieler sind jeweils vier Rochade-Situationen möglich:
  - Keine Rochade möglich
  - Nur königsseitige Rochade möglich
  - Nur damenseitige Rochade möglich
  - Beidseitige Rochade möglich
- 4) **En-Passant-Situation**  $E$ :
  - Es kann nicht En-Passant geschlagen werden
  - Es kann En-Passant-geschlagen werden (In diesem Fall ist zu unterscheiden, welcher Bauer kann En-Passant geschlagen werden kann.)
- 5) **Zugzähler**: Anzahl der Züge seit dem letzten Bauernzug oder dem letzten Zug, in dem geschlagen wurde. Je nachdem welcher Fall zuletzt eingetreten ist. Dies ist dient der 50-Züge-Regel.
- 6) **Wiederholungsanzahl**: Anzahl wie oft diese Figurenverteilung im Schachspiel vorgelegen hat. Ist die Wiederholungsanzahl drei, endet das Spiel in einem Remis.

Shannon selbst definiert eine Schachposition in [4] so, dass sie die Informationen 1) bis 5) enthält, obwohl er - wie wir in Kapitel zwei sehen werden - lediglich Informationen aus 1) bei seiner Schätzung berücksichtigt. Chinchalkar und Tromp ignorieren 5) und 6) komplett. Bei Chinchalkar ist dies in [1] auf S. 181 zu finden. Bei Tromp folgt dies daraus, dass er Urpositionen - Schachpositionen mit Zusatzinformationen, auf die wir in Kapitel sechs genauer eingehen werden - durch eine modifizierte FEN-Notation eindeutig beschreibt und durch diese Modifikation die Stelle für den Zugzähler in der FEN-Notation überschreiben wird. Somit haben Urpositionen und damit auch Schachpositionen nach seiner Definition keinen Zugzähler. Informationen über die Wiederholungszahl sind in

der FEN-Notation gar nicht erst vorhanden und damit auch nicht Teil der relevanten Informationen für eine Schachposition nach seiner Definition. In dieser Arbeit orientieren wir uns an den moderneren Autoren, sodass eine Schachposition wie folgt definiert ist:

**Definition 1.1 (Schachposition)** *Eine Schachposition  $S$  ist ein Tupel  $(F, Z, R, E)$ , wobei  $F$  die Figurenverteilung,  $Z$  das Zugrecht,  $R$  die Rochade-Situation und  $E$  die En-Passant-Situation ist. Zwei Schachpositionen  $S$  und  $S'$  sind verschieden, wenn sie sich in einer dieser vier Komponenten unterscheiden.*

**Definition 1.2 (Legale und Illegale Schachposition)** *Eine Schachposition  $S$  heißt legal, wenn sie in einem Schachspiel erreicht werden kann, andernfalls heißt sie illegal.*

Da wir im Laufe dieser Bachelorarbeit viel zu erklären haben, definieren wir bereits jetzt einige Begriffe, die uns dabei helfen werden:

- **Capture** ist der Begriff dafür, dass eine Figur geschlagen wird.
- Die Variable **caps** beschreibt die Anzahl an Captures.
- **Piece** ist ein Sammelbegriff für die Figuren Springer, Läufer, Turm und Dame.
- **Promotion** ist der Begriff für die Umwandlung eines Bauern in ein Piece, wenn er die gegenüberliegende Spielfeldseite erreicht.
- Eine **Figurenkombination** ist ein Tupel, das für jeden Figurentyp jeder Farbe die Anzahl enthält, wie oft dieser Figurentyp auf dem Schachbrett steht.
- Eine **einseitige Figurenkombination** ist eine Figurenkombination, die nur Informationen über die Figurentypen einer Farbe enthält.
- **Legale Figurenkombination** ist der Begriff für eine Figurenkombination, die durch ein Schachspiel erreicht werden kann.
- Die Variablen **wB**, **wD**, **wT**, **wL** und **wS** beschreiben (in dieser Reihenfolge) die Anzahl weißer Bauern, Damen, Türme, Läufer und Springer auf dem Spielfeld.
- Die Variablen **sB**, **sD**, **sT**, **sL** und **sS** beschreiben (in dieser Reihenfolge) die Anzahl schwarzer Bauern, Damen, Türme, Läufer und Springer auf dem Spielfeld.
- Die Variablen **wprom** bzw. **sprom** beschreiben die Anzahl der stattgefundenen weißen bzw. schwarzen Promotionen.

- Die Variablen **wpx** bzw. **bpx** beschreiben die Anzahl an geschlagenen weißen bzw. schwarzen Bauern.
- **Bauernterritorium** ist der Sammelbegriff für die 48 Felder der Reihen zwei bis sieben, da dies die einzigen Felder sind, auf denen Bauern stehen können.

## 2 Shannons Schätzung

Im ersten Teil dieses Kapitels folgen wir Shannons Ausführungen in [4]. In der zweiten Hälfte werden wir erklären, wie Shannon auf seinen Schätzwert kommt und selbigen einordnen. Die zweite Hälfte ab der Erklärung der Wahl des Multinomialkoeffizienten ist meine Eigenleistung.

Shannon beweist in seiner Arbeit, dass es keinen perfekt Schach spielenden Computer gibt. Dies gelingt ihm, indem er zeigt, dass es nur zwei Möglichkeiten gibt einen perfekt Schach spielenden Computer zu programmieren und deren Umsetzung aufgrund der immensen Anzahl an Schachpositionen und Schachspielen zwar theoretisch möglich, aber niemals praktisch realisierbar ist.

Die erste Möglichkeit ist ein Programm, das einen Minimax-Algorithmus mit unendlicher Tiefe ausführt und sich dann für den besten Zug entscheidet. Für dieses Vorgehen muss das Programm jedoch zu Spielbeginn alle möglichen Schachspiele durchgehen und bewerten. Shannon schätzt, dass es ungefähr  $10^{120}$  verschiedene Schachspiele gibt, für deren Auswertung ein Computer der damaligen Zeit ungefähr  $10^{90}$  Jahre brauchte. Obwohl die Computer heute deutlich schneller sind, ist die Dauer auch heute noch deutlich zu lang.

Die zweite Möglichkeit ist ein Verzeichnis, in dem für jede mögliche Schachposition der bestmögliche Antwortzug gespeichert ist. Shannon erklärt, dass der Aufbau eines solchen Verzeichnisses theoretisch möglich sei, indem man starke Schachspieler befragt und Schachpositionen durch den oben beschriebenen Minimax-Algorithmus analysiert. Das Problem bei dieser Möglichkeit ist die von Shannon geschätzte Anzahl der möglichen Schachpositionen von ungefähr  $10^{43}$ . Auch diese ist wesentlich zu groß und macht die Umsetzung eines solchen Verzeichnisses in der Realität ebenfalls unmöglich.

Shannon kommt auf die Zahl  $10^{43}$  durch den folgenden Multinomialkoeffizienten, welchen er auf  $10^{43}$  aufrundet.

$$\binom{64}{32, 8, 8, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1} \approx 4,6347 \cdot 10^{42}$$

Shannon erklärt nicht, warum er diesen Multinomialkoeffizienten verwendet, jedoch berechnet genau dieser Multinomialkoeffizient die Anzahl an Möglichkeiten die Figuren, die zu Spielbeginn auf dem Schachbrett stehen auf den 64 Feldern zu verteilen. Es ist stark davon auszugehen, dass er deshalb diesen Multinomialkoeffizienten als Schätzwert für die Anzahl aller möglichen Schachpositionen verwendet.

Hier sieht man gut, dass Shannon für seine Schätzung lediglich Unterschiede in der Figurenverteilung verwendet und somit mögliche Unterschiede im Zugrecht, in der En-Passant-Situation, in der Rochade-Situation und im Zugzähler ignoriert. Wobei man argumentieren kann, dass das Aufrunden auf  $10^{43}$  mehr als einer Verdopplung von  $4,6347 \cdot 10^{42}$  entspricht und somit als Einbeziehung des Zugrechts gewertet werden könne. Shannon verzichtet auf die Einbeziehung

weiterer Informationen, da es für sein Argument irrelevant ist, ob es  $10^{43}$ ,  $10^{50}$  oder  $10^{100}$  verschiedene legale Schachpositionen gibt. Denn alle Zahlen beschreiben eine Anzahl von Schachpositionen, die von keinem Computer in angemessener Zeit bewertet werden kann.



Abb. 1: Diese Schachposition ist illegal, da der Läufer auf f4 nur dort stehen kann, wenn einer der Bauern auf b2 oder d2 weggezogen wurde. Diese Schachposition wird bei Shannons Schätzung jedoch mitgezählt.

Shannons Schätzung ist - wie wir in Kapitel sechs sehen werden - für den geringen Rechenaufwand eine sehr gute Approximation der Anzahl an legalen Schachpositionen. Sein Schätzwert ist aber keine obere Schranke, da die En-Passant-Situation, die Rochade-Situation und alle Figurenverteilungen, bei denen mindestens eine Promotion oder mindestens ein Capture stattgefunden haben, nicht berücksichtigt werden. Darüber hinaus ist sein Schätzwert auch keine untere Schranke, da illegale Schachpositionen (s. Abb. 1) ohne eine obere Schranke an deren Anzahl mitgezählt werden.



### 3 Chinchalkars Lösung

In seiner Arbeit [1] findet Shirish Chinchalkar mithilfe von Computerprogrammen eine obere Schranke in der Größenordnung von  $1,77894 \cdot 10^{46}$ .

Die Datei `npos ORIG.c` des Github-Posts [2] von William Entriken enthält den Programmcode, den Chinchalkar verwendet hat. Im folgenden Kapitel werden wir die von Chinchalkar in [1] verwendeten Konzepte erklären und sie mit den Codeanweisungen aus [2] vergleichen. Sofern wir uns nicht auf den Programmcode beziehen, sind Chinchalkars Ergebnisse immer in Quelle [1] zu finden.

Chinchalkar wählt einen zweischrittigen Lösungsweg. Im ersten Schritt errechnet er mit Hilfe von notwendigen Bedingungen für legale Schachpositionen eine möglichst kleine Menge  $X$  von Figurenkombinationen, in der alle legalen Figurenkombinationen enthalten sind. Im zweiten Schritt berechnet er für jede dieser Figurenkombinationen eine obere Schranke an die Anzahl der Möglichkeiten, die Figuren, die durch diese Figurenkombination beschrieben werden, auf dem Schachbrett zu verteilen. Abschließend addiert er alle oberen Schranken und erhält damit eine obere Schranke an alle legalen Schachpositionen.

#### 3.1 Berechnung der Figurenkombinationen

Chinchalkar erwähnt in [1] einige unbewiesene notwendige Bedingungen, mit denen er die Menge  $X$  auf 58.084.310 Elemente einschränkt. Wir werden in diesem Kapitel alle Bedingungen aus Chinchalkars Arbeit als Sätze formulieren und beweisen. Abschließend werden wir jeweils die Umsetzung der Sätze im Code betrachten. Alle Beweise dieses Unterkapitels, alle Lemmata und die Konzepte des Spaltenwechsels und der Schlag- und Opfer-Spalte sind meine Eigenleistung. Um die Bedingungen von Chinchalkar beweisen zu können, führen wir einige Begriffe ein und beweisen ein Lemma.

**Definition 3.1 (Freie Spalte und Blockierte Spalte)** *Eine Spalte heißt blockiert, wenn sich in dieser Spalte zwei Bauern verschiedener Farben so blockieren, dass sie keine Promotion durchführen können. Eine Spalte heißt frei, wenn sie nicht blockiert ist.*

**Definition 3.2 (Spaltenwechsel)** *Ein Spaltenwechsel passiert, wenn ein Bauer eine Figur schlägt und dadurch die Spalte wechselt.*

**Definition 3.3 (Schlag-Spalte und Opfer-Spalte)** *Wenn ein Bauer eine Figur schlägt, ist die Spalte, in der sich der Bauer befand, die Schlag-Spalte. Die Spalte, in der sich die geschlagene Figur befand, ist die Opfer-Spalte.*

Wir sagen, dass Bauern in freien Spalten Promotionen ohne weiteres Capture

durchführen können, da lediglich Figuren aus dem Weg gezogen werden müssen, aber keine Figuren mehr geschlagen werden müssen, damit diese Bauern eine Promotion durchführen können.

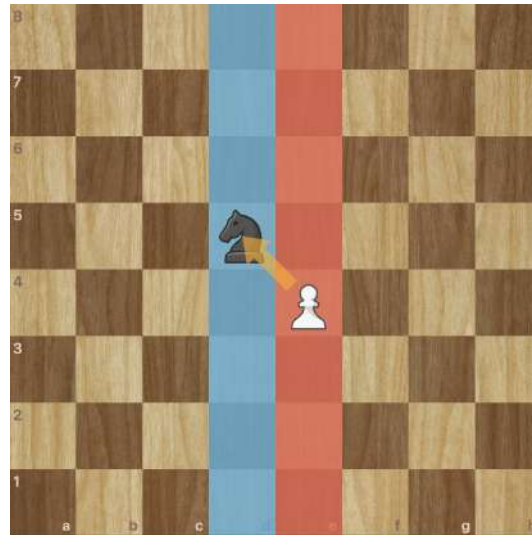


Abb. 2: Der weiße Bauer schlägt den schwarzen Springer. Die Schlag-Spalte ist rot markiert und die Opfer-Spalte blau.

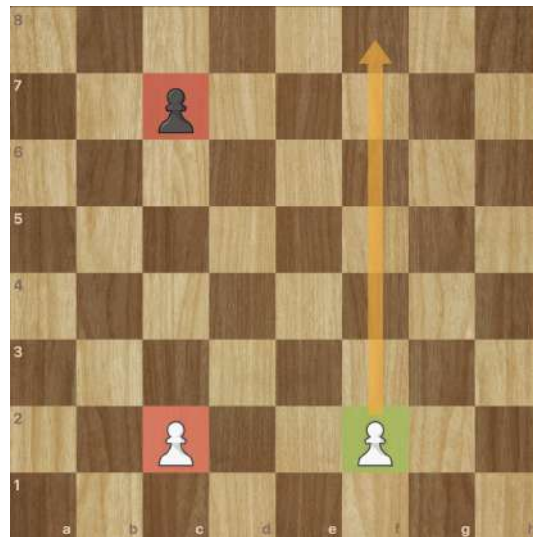


Abb. 3: Die beiden Bauern in Spalte c blockieren sich, während sich der Bauer in Spalte f in einer freien Spalte befindet und somit eine Promotion ohne weiteres Capture durchführen kann.

**Lemma 3.4** *Es gilt:*

- a) *In einem Zug ohne Capture wird keine Spalte frei.*
- b) *Gab es noch kein Capture, kann keine Promotion stattfinden.*
- c) *In einem Zug, in dem ein Bauer ein Piece schlägt, wird maximal die Schlag-Spalte frei.*
- d) *In einem Zug, in dem ein Bauer geschlagen wird, werden maximal die Schlag- und Opfer-Spalte frei.*
- e) *In einem Zug, in dem ein Piece ein anderes Piece schlägt, wird keine Spalte frei.*

**Beweis** Wir werden die einzelnen Aussagen nacheinander beweisen. In den nachfolgenden Abbildungen sind die verschiedenen Fälle veranschaulicht.

Beweis a): Freie Spalten können nur durch Spaltenwechsel und damit durch Captures entstehen. In einem Zug ohne Capture kann somit keine Spalte frei werden.

Beweis b): Da ein Bauer nur dann eine Promotion durchführen kann, wenn er sich in einer freien Spalte befindet und zu Spielbeginn keine Spalte frei ist, kann es keine Promotionen vor dem ersten Capture geben.

Beweis c): Schlägt ein Bauer ein Piece, wird dadurch maximal die Schlag-Spalte frei. Denn die Opfer-Spalte kann nicht frei werden, da kein Bauer aus ihr verschwindet. Alle anderen Spalten sind vom Capture nicht betroffen.

Beweis d): Schlägt ein Bauer einen anderen Bauern, werden maximal die Schlag- und die Opfer-Spalte frei, da alle anderen Spalten vom Capture nicht betroffen sind. Falls ein Bauer von einer anderen Figur geschlagen wird, wird analog zum Beweis von c) lediglich die Opfer-Spalte frei. Daraus folgt, dass maximal die Schlag- und die Opfer-Spalte frei wird, wenn ein Bauer geschlagen wird.

Beweis e): Wenn ein Piece ein anderes Piece schlägt, wurde kein Bauer bewegt, daher kann es keinen Spaltenwechsel geben. Da es keinen Spaltenwechsel gibt und kein Bauer geschlagen wurde, kann keine Spalte frei geworden sein.  $\square$

In den nächsten vier Abbildungen zeigen die Spalten a und b jeweils die Situation vor dem Capture und die Spalten g und h die Situation danach. Bauern in freien Spalten sind grün markiert und Bauern in blockierten Spalten haben eine rote Markierung. Die beiden blauen Pfeile symbolisieren, dass die Situation rechts aus dem Capture resultiert und die orangen Pfeile zeigen an, welche Figuren am Capture beteiligt sind bzw. welche Figuren eine Promotion durchführen können.



Abb. 4: Schlägt ein Bauer ein Piece, wird dadurch maximal die Schlag-Spalte frei.



Abb. 5: Schlägt ein Bauer einen anderen Bauern, werden dadurch maximal die Schlag-Spalte und die Opfer-Spalte frei.



Abb. 6: Schlägt ein Piece einen Bauern, wird dadurch maximal die Opfer-Spalte frei.



Abb. 7: Schlägt ein Piece ein anderes Piece, wird gar keine Spalte frei.

Nun können wir einige von Chinchalkars Bedingungen als Satz formulieren und beweisen. Chinchalkar erwähnt in [1] lediglich die notwendigen Bedingungen 1) bis 4) für legale Schachpositionen, wobei die Aussage 4) bei ihm nur für Bauern gilt, die von anderen Bauern geschlagen wurden. Ich habe die Aussage um Bauern erweitert, die von Pieces geschlagen wurden. 5) ist ebenfalls eine Ergänzung von mir.

**Satz 3.5** *Es gilt:*

- 1) *Pro Seite muss genau ein König auf dem Feld stehen*
- 2) *Es gilt:*
  - (a)  $w_{\text{prom}} \leq 8 - wB$
  - (b)  $s_{\text{prom}} \leq 8 - sB$
- 3) *Für jedes Piece, das von einem Bauern geschlagen wird, kann jede Seite maximal eine Promotion durchführen*
- 4) *Für jeden Bauer, der geschlagen wird, kann die Seite, deren Bauer geschlagen wurde, maximal eine Promotion und die andere Seite maximal zwei Promotionen durchführen.*
- 5) *Es gibt keine anderen Möglichkeiten, durch die Promotionen möglich werden, als durch die in 3) und 4) beschriebenen Captures.*

**Beweis** 1) und 2) sind direkte Ableitungen aus den Spielregeln, da genau ein König pro Seite immer auf dem Feld stehen muss und es nicht mehr Promotionen einer Seite geben kann, als es Bauern dieser Seite gibt, die das Spielfeld verlassen haben. 5) wurde durch die Punkte a), b) und e) von Lemma 3.4 bewiesen.

Sei  $BxP$  ein Capture, bei dem ein Bauer ein Piece schlägt,  $BxB$  ein Capture, bei dem ein Bauer einen anderen Bauern schlägt und  $PxB$  ein Capture, bei dem ein Piece einen Bauern schlägt. Sei  $wprom_X$  die Anzahl der weißen Promotionen, die durch ein Capture  $X$  ermöglicht werden und  $sprom_X$  die Anzahl der schwarzen Promotionen, die durch ein Capture  $X$  ermöglicht werden. Sei  $x_s$  die Maximalanzahl an schwarzen Promotionen, die nach 3) und 4) in der Spielsituation erlaubt sind und  $x_w$  die Maximalanzahl an weißen Promotionen, die nach 3) und 4) in der Spielsituation erlaubt sind.

Für den Beweis von 3) und 4) verwenden wir einen Widerspruchsbeweis. Wir nehmen an, dass es eine Spielsituation gibt, in der  $\beta := wprom > x_w \vee sprom > x_s$  gilt. Dann muss es ein Capture geben, nach dem  $\beta$  zum ersten Mal gilt. Sei  $A$  das Capture, nach dem  $\beta$  zum ersten Mal eingetreten ist. Wir nehmen o. B. d. A. für  $A$  an, dass ein schwarze Figur die schlagende Figur ist. Dann muss für  $A$  gelten:

- Ist  $A$  ein  $BxP$ , muss gelten:  $wprom_A > 1 \vee sprom_A > 1$
- Ist  $A$  ein  $BxB$ , muss gelten:  $wprom_A > 1 \vee sprom_A > 2$
- Ist  $A$  ein  $PxB$ , muss gelten:  $wprom_A > 1 \vee sprom_A > 2$

Falls dies nicht gilt, würde entweder  $\beta$  nicht gelten oder  $A$  wäre nicht das erste Capture, nach dem  $\beta$  gilt.

Fall 1:  $A$  ist ein  $BxP$ :

Nach Lemma 3.4 wird maximal die Schlag-Spalte von  $A$  (wir nennen sie  $S$ ) frei. Falls  $S$  vor  $A$  frei war, wird durch  $A$  keine Spalte frei und somit entstehen durch  $A$  überhaupt keine Promotionen. Falls  $S$  vor  $A$  blockiert war, wird  $S$  nur dann frei, wenn keine anderen schwarzen Bauern in  $S$  stehen. Somit ist der schlagende schwarze Bauer der einzige schwarze Bauer, der durch  $A$  in eine freie Spalte gelangen kann und  $sprom_A \leq 1$  gilt.

Dies bedeutet, dass in der Spielsituation  $sprom > x_s$  nicht gelten kann. Angenommen  $sprom > x_s$  würde gelten, dann würde  $sprom > x_s$  auch bereits vor  $A$  gelten, da  $sprom_A \leq 1$  ist und durch ein Capture vom Typ  $BxP$  nach 3) ebenfalls eine schwarze Promotion möglich wird. Dies widerspricht jedoch der Aussage, dass  $\beta$  nach  $A$  zum ersten Mal gilt.

Da  $sprom_A > 1$  nicht gilt, muss  $wprom_A > 1$  gelten. Wir nehmen an, dass  $wprom_A = n \geq 2$  gilt. Da  $S$  die einzige Spalte ist, die durch  $A$  frei wird, müssen  $n$  weiße Bauern in  $S$  stehen, damit  $wprom_A = n$  gelten kann. Damit  $n$  weiße Bauern in  $S$  stehen, muss es mindestens  $n - 1$  Spaltenwechsel nach  $S$  und somit mindestens  $n - 1$  Captures  $C_1, \dots, C_l$  mit  $l \geq n - 1$  gegeben haben, bei

denen weiße Bauern eine Figur in  $S$  geschlagen haben. Für ein solches Capture  $B \in \{C_1, \dots, C_l\}$  gibt es nun zwei Möglichkeiten:

Fall 1.1:  $B$  ist ein  $BxP$ :

Da Weiß schlägt, sind alle Promotionen, die in der Schlag-Spalte von  $B$  durch  $B$  möglich werden schwarze Promotionen. Da die Opfer-Spalte von  $B$  die Spalte  $S$  ist, die erst durch  $A$  frei wird, gilt  $wprom_B = 0$ . Demnach wird durch  $B$  eine weiße Promotion weniger möglich als für ein Capture vom Typ  $BxP$  nach 3) erlaubt ist.

Fall 1.2:  $B$  ist ein  $BxB$ :

Es wird maximal die Schlag-Spalte von  $B$  frei, da die Opfer-Spalte von  $B$  die Spalte  $S$  ist und diese erst durch  $A$  frei wird. Da Weiß schlägt, sind alle Promotionen, die in der Schlag-Spalte von  $B$  durch  $B$  möglich werden, schwarze Promotionen. Es gilt also  $wprom_B = 0$ . Damit werden durch  $B$  sogar zwei weiße Promotionen weniger möglich, als durch eine Capture vom Typ  $BxB$  nach 4) erlaubt sind.

Somit machen  $n-1$  dieser Captures mindestens  $n-1$  weiße Promotionen weniger möglich, als eigentlich durch 3) und 4) erlaubt wären. Da  $A$  nur  $n-1$  weiße Promotionen mehr möglich macht als für ein Capture vom Typ  $BxP$  nach 3) erlaubt sind, werden durch die Captures  $A, C_1, \dots, C_l$  nicht mehr weiße Promotionen ermöglicht als nach den Regeln von 3) und 4) erlaubt sind.

Daraus folgt, dass die  $wprom > x_w$  nicht gelten kann. Denn würde  $wprom > x_w$  gelten, hätte es auch bereits vor den Captures  $A, C_1, \dots, C_l$  gegolten. Dies widerspricht jedoch der Aussage, dass  $\beta$  nach  $A$  erstmals gilt.

Es ist also nicht möglich, dass  $A$  ein Capture vom Typ  $BxP$  ist.

Fall 2:  $A$  ist ein  $BxB$ :

Es werden maximal die Schlag-Spalte von  $A$  (wir nennen sie weiterhin  $S$ ) und die Opfer-Spalte von  $A$  (wir nennen sie  $O$ ) frei. Es sind zwei Fälle möglich.

Fall 2.1:  $sprom_A > 2$ :

Sei  $sprom_A = m \geq 3$ . Dann muss es  $m$  schwarze Bauern in  $O$  geben. Dafür sind mindestens  $m-2$  Spaltenwechsel nach  $O$  nötig. Der Beweis kann von hier aus analog zu Fall 1 erfolgen. Schließlich folgt, dass  $sprom > x_s$  nicht, gilt oder bereits vor  $A$  gegolten haben muss.

Fall 2.2:  $wprom_A > 1$ :

Dieser Fall kann analog zu Fall 2.1 bewiesen werden.

Damit ist es nicht möglich, dass  $A$  ein Capture vom Typ  $BxB$  ist.

Fall 3:  $A$  ist ein  $PxB$ :

Durch  $PxB$  wird maximal die Opfer-Spalte von  $A$  frei (wir nennen sie  $O$ ), da kein Bauer die Schlag-Spalte verlässt. Da wir davon ausgehen, dass die schlagende Figur eine schwarze Figur ist, wird  $O$  frei, indem ein weißer Bauer geschlagen wird. Sollte  $O$  durch  $A$  frei werden, ist  $wprom_A = 0$ , sollte  $O$  durch  $A$  nicht



frei werden, gilt ebenfalls  $wprom_A = 0$ . Weil  $wprom_A > 1$  nicht gilt, muss  $sprom_A > 2$  gelten. Sei  $sprom_A = m \geq 3$ . Der Beweis kann von hier aus analog zu Fall 2.1 erfolgen. Schließlich folgt, dass  $sprom > x_s$  nicht gilt oder bereits vor  $A$  gegolten haben muss.

Damit ist es nicht möglich, dass  $A$  ein Capture vom Typ  $PxB$  ist. Also gibt es kein Capture  $A$ , nach dem  $\beta$  erstmals gilt und da  $\beta$  nicht zu Spielbeginn gilt, gibt es keine Spielsituation, in der  $wprom > x_w \vee sprom > x_s$  gilt.  $\square$

**Bemerkung** Im Beweis wird die Aussage verwendet, dass die Spalte  $S$  blockiert sein muss, da sie erst später durch das Capture  $A$  frei wird. Diese Formulierung hat die Situation etwas vereinfacht dargestellt. Es ist natürlich möglich, dass eine Spalte  $S$  frei ist, dann durch einen Spaltenwechsel blockiert wird und schließlich durch ein Capture  $A$  erneut frei wird. Jedoch zählen alle Bauern, die sich in dieser Spalte befinden haben, als sie zum ersten Mal frei war, nicht zu Promotionen, die durch das Capture  $A$  ermöglicht werden, da die Bauern bereits einmal die Möglichkeit hatten eine Promotion durchzuführen. Da wir im Beweis davon ausgegangen sind, dass  $A$   $n$  weiße Promotionen ermöglicht, müssen die  $n - 1$  zusätzlichen weißen Bauern zu einem Zeitpunkt in die Spalte  $S$  gelangt sein, an dem  $S$  blockiert war.

Aus Satz 3.5 leiten wir die notwendigen Bedingungen ab, die Chinchalkar in seinem Programmcode verwendet hat.

**Satz 3.6** *Sei*

- $\triangleright$   $wpcx$  die Anzahl der weißen Pieces, die geschlagen wurden
- $\triangleright$   $bpcx$  die Anzahl der schwarzen Pieces, die geschlagen wurden

*Dann gilt*

- a)  $wprom \leq 2 \cdot bpx + wpcx + bpcx + wpx$
- b)  $sprom \leq 2 \cdot wpx + wpcx + bpcx + bpx$

**Beweis** a) und b) sind jeweils analoge Aussagen für die beiden Seiten, deshalb reicht es a) zu zeigen. Wie wir in Satz 3.5 gesehen haben, kann man für jeden geschlagenen schwarzen Bauern maximal zwei weiße Promotionen, also insgesamt  $2 \cdot bpx$  weiße Promotionen durchführen.  $wpcx + bpcx$  beschreibt die Anzahl der geschlagenen Pieces. Für jedes geschlagene Piece, kann man nach Satz 3.5 maximal eine weiße Promotion, also insgesamt  $wpcx + bpcx$  weiße Promotionen durchführen. Für jeden geschlagenen weißen Bauern kann man nach Satz 3.5 maximal eine weiße Promotion, also insgesamt  $wpx$  weiße Promotionen durchführen. Da dies laut Satz 3.5 die einzigen Möglichkeiten sind, wie

weiße Promotionen entstehen können, ist  $2 \cdot bpx + wpcx + bpcx + wpx$  eine obere Schranke an  $wprom$ .  $\square$

Einige grundlegende Eigenschaften über die Figurenanzahlen fassen wir im folgenden Lemma zusammen.

**Lemma 3.7** *Es gilt:*

- $\triangleright wB, sB \in \{0, \dots, 8\}$
- $\triangleright wD, sD \in \{0, \dots, 9\}$
- $\triangleright wS, sS, wL, sL, wT, sT \in \{0, \dots, 10\}$
- $\triangleright wB + wD + wS + wL + wT + 1 \leq 16$
- $\triangleright sB + sD + sS + sL + sT + 1 \leq 16$

**Beweis** Alle Aussagen folgen direkt aus den Schachregeln.  $\square$

Betrachten wir nun die Umsetzung unserer Ergebnisse in der *npos ORIG.c*-Datei von [2]. Chinchalkar geht in der main-Funktion mithilfe der for-Schleifen in den Zeilen 215 bis 246 die Anzahlen an Bauern, Damen, Türmen, Springern und Läufern und Bauern durch, die eine Seite haben kann. Mit der if-Bedingung in Zeile 225f stellt er sicher, dass 2) aus Satz 3.5 erfüllt ist. Sofern dies der Fall ist, wird die einseitige Figurenkombination im Array *pieces* gespeichert (Z. 227-234). Das Array *pieces* beschreibt somit die Menge aller möglichen einseitigen Figurenkombinationen. Die for-Schleifen, die in den Zeilen 256 und 269 beginnen, durchlaufen sowohl für Weiß als auch für Schwarz das Array *pieces* und gehen damit jede Kombination zweier einseitiger Figurenkombinationen durch. In den Zeilen 286 bis 289 werden die Bedingungen aus Satz 3.6 geprüft, wobei  $nWcaptures = wpx + wpcx$  und  $nBcaptures = bpx + bpcx$  ist. Erfüllt eine Figurenkombination alle Kriterien, ist sie ein Element von  $X$  und die Variable *nsets* wird in Zeile 303 um eins erhöht. *nsets* beschreibt damit die Mächtigkeit von  $X$ . Als Ergebnis erhält man, dass die Menge  $X$  58.084.310 verschiedene Elemente besitzt.

### 3.2 Platzierung der Figurenkombinationen

In diesem Abschnitt verfolgen wir Chinchalkar, wie er in [1] für jede Figurenkombination  $x$  der Menge  $X$  eine Zahl  $\Theta_x$  berechnet.  $\Theta_x$  ist eine obere Schranke an die Anzahl der Möglichkeiten die Figurenkombination  $x$  auf dem Schachbrett zu verteilen und berücksichtigt die Rochade-Situationen, die En-Passant-Situationen und das Zugrecht. Durch das Aufsummieren der oberen Schranken erhält er eine obere Schranke  $O$  an die Anzahl der legalen Schachpositionen. Wenn wir davon sprechen, dass eine obere Schranke die En-Passant-Situationen, die Rochade-Situationen und das Zugrecht berücksichtigt, bedeutet dies, dass

die Elemente aller Fälle, die in den En-Passant-Situationen, in den Rochade-Situationen und im Zugrecht auftreten können, mitgezählt werden.

Für jede Figurenkombination verteilt Chinchalkar zunächst die Bauern auf dem Schachbrett. Er formuliert in [1] den folgenden Satz, der die obere Schranke  $W_{Bauer}$  findet.

**Satz 3.8** *Sei*

- $i$  die Anzahl der weißen Bauern in der vierten Reihe
- $j$  die Anzahl der weißen Bauern in der fünften Reihe
- $l$  die Anzahl der schwarzen Bauern in der vierten Reihe
- $k$  die Anzahl der schwarzen Bauern in der fünften Reihe
- $p = \min(sB, 8 - j)$
- $q = \min(sB - k, 8 - i)$

Dann ist  $W_{Bauer}$  eine obere Schranke an die Anzahl an Möglichkeiten  $wB$  weiße Bauern und  $sB$  schwarze Bauern auf dem Schachbrett zu verteilen, die die En-Passant-Situation und das Zugrecht berücksichtigt, wobei

$$\begin{aligned}
 &\text{➤ } W_{Bauer} = \sum_{i=0}^{wB} \sum_{j=0}^{wB-i} \sum_{k=0}^p \sum_{l=0}^q W_{ij} \cdot B_{ijkl} \cdot E_{ijkl} \\
 &\text{➤ } W_{ij} = \binom{8}{i} \cdot \binom{8}{j} \cdot \binom{32}{wB-i-j} \\
 &\text{➤ } B_{ijkl} = \binom{8-j}{k} \cdot \binom{8-i}{l} \cdot \binom{32-(wB-i-j)}{sB-k-l} \\
 &\text{➤ } E_{ijkl} = (1 + \min(2l, i) + 1 + \min(2j, k))
 \end{aligned}$$

**Beweis** Alle Erklärungen bis auf die Erklärungen der Terme  $\min(2j, k)$  und  $\min(2l, i)$  sind meine Eigenleistung.  $W_{ij}$  beschreibt die Anzahl an Möglichkeiten

$wB$  weiße Bauern so auf dem Schachbrett zu verteilen, dass  $i$  weiße Bauern in Reihe vier und  $j$  weiße Bauern in Reihe fünf stehen. Nachdem die  $wB$  weißen Bauern derartig verteilt wurden, beschreibt  $B_{ijkl}$  die Anzahl an Möglichkeiten die  $sB$  schwarzen Bauern so auf den restlichen Feldern zu verteilen, dass  $k$  schwarze Bauern in Reihe fünf und  $l$  schwarze Bauern in Reihe vier stehen. In den Binomialkoeffizienten steht eine 32, da den Bauern 32 Felder außerhalb der vierten und fünften Reihe zur Verfügung stehen.

In  $E_{ijkl}$  verarbeitet Chinchalkar sowohl das Zugrecht als auch die möglichen En-Passant-Situationen dieser Bauernverteilung. Der Term  $1 + \min(2j, k)$  beschreibt die Maximalanzahl an En-Passant-Situationen, wenn Weiß am Zug ist und der Term  $1 + \min(2l, i)$  die Maximalanzahl an En-Passant-Situationen, wenn

Schwarz am Zug ist. Es liegen zwei verschiedene En-Passant-Situationen vor, wenn verschiedene Bauern En-Passant geschlagen werden können. Wir nehmen an, dass Weiß am Zug ist. Der Term  $1 + \min(2l, i)$  kann analog bewiesen werden. Für die Berechnung einer oberen Schranke an die Anzahl der En-Passant-Situationen muss Chinchalkar herausfinden, wie viele verschiedene schwarze Bauern in dieser Bauernverteilung maximal En-Passant geschlagen werden können. Stehen  $j$  weiße Bauern in der fünften Reihe, können maximal  $2j$  verschiedene schwarzen Bauern En-Passant geschlagen werden, da jeder weiße Bauer nur nach links oder nach rechts schlagen kann. Darüber hinaus können nur die schwarzen Bauern En-Passant geschlagen werden, die sich in der fünften Reihe befinden. Demnach ist  $\min(2l, i)$  eine obere Schranke an die Anzahl der En-Passant-Situationen, wenn Weiß En-Passant schlagen kann. Eine weitere En-Passant-Situation liegt vor, wenn nicht En-Passant geschlagen werden kann. Somit ist  $\min(2j, k) + 1$  eine obere Schranke an die Anzahl an En-Passant-Situationen wenn Weiß am Zug ist.

Summiert man das Produkt  $W_{ij} \cdot B_{ijkl} \cdot E_{ijkl}$  über alle möglichen Werte auf, die  $i$ ,  $j$ ,  $k$  und  $l$  annehmen können, erhält man eine obere Schranke an die Anzahl der Möglichkeiten  $wB$  weiße Bauern und  $sB$  schwarze Bauern auf dem Schachbrett zu verteilen, die die En-Passant-Situationen und das Zugrecht berücksichtigt.  $\square$

Obwohl  $W_{Bauer}$  die Anzahl an möglichen Bauernverteilungen eingeschränkt hat, macht Chinchalkar darauf aufmerksam, dass  $W_{Bauer}$  die Anzahl an Bauernpositionen auf zwei Arten überschätzt:

1. Es sind nicht alle Bauernpositionen möglich. (s. Abb. 8)
2. Es sind nicht alle En-Passant-Situationen möglich (s. Abb. 9)



Abb. 8: Es ist unmöglich, dass weiße Bauern gleichzeitig auf a2, a3 und b2 stehen, da entweder der Bauer von a2 oder b2 auf a3 gezogen sein muss.



Abb. 9: Es ist unmöglich, dass der weiße Bauer auf e5 den schwarzen Bauern auf h5 En-Passant schlägt. Diese Möglichkeit zählt  $W_{Bauer}$  jedoch mit.

Um möglichst wenige illegale Bauernverteilungen mitzuzählen, erwähnt Chin-chalkar eine Möglichkeit  $W_{Bauer}$  zu verbessern, indem man basierend auf der Anzahl der Captures herausfindet, wie viele Spalten blockiert sind. Obwohl er in seiner Arbeit nicht weiter auf diese Idee eingeht, ist die Umsetzung in den Zeilen 137 bis 152 der *npos ORIG.c*-Datei zu finden. Wir werden die Berechnung als Satz 3.9 formulieren und beweisen.

**Satz 3.9** Seien  $x \in \{1, \dots, 8\}$  Spalten blockiert, dann ist  $M$  eine obere Schranke an die Anzahl der Verteilungen von  $wB$  weißen Bauern und  $sB$  schwarzen Bauern auf dem Schachbrett, die die En-Passant-Situationen und das Zugrecht berücksichtigt, wobei

$$M = \binom{8}{x} \cdot 15^x \cdot \binom{48-2x}{wB-x} \cdot \binom{48-wB-x}{sB-x} \cdot y$$

mit  $y = (1 + \min(2 \cdot wB, sB) + 1 + \min(2 \cdot sB, wB))$

Für  $x = 0$  soll weiterhin  $W_{\text{Bauer}}$  aus Satz 3.8. verwendet werden. Somit gilt für diesen Fall  $M = W_{\text{Bauer}}$ .

**Beweis** Die Erklärungen dieses Beweises sind eine Kombination aus den Kommentaren des Programmcodes und meiner Eigenleistung.

Seien  $x \in \{1, \dots, 8\}$  Spalten blockiert. Dann gibt es maximal  $\binom{8}{x}$  verschiedene Kombinationen von blockierten Spalten. Darüber hinaus gibt es in jeder blockierten Spalte maximal 15 Möglichkeiten, wie sich der schwarze und der weiße Bauer blockieren können. Diese Möglichkeiten sind (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (3, 4), (3, 5), (3, 6), (3, 7), (4, 5), (4, 6), (4, 7), (5, 6), (5, 7) und (6, 7), wobei die erste Komponente jeweils die Reihe des weißen Bauern und die zweite Komponente jeweils die Reihe des schwarzen Bauern beschreibt (s. Abb. 10 und 11). In einer blockierten Spalte können auch mehr als nur ein weißer und ein schwarzer Bauer stehen, diese zusätzlichen Bauern betrachten wir gleich, wenn wir die restlichen Bauern auf dem Schachbrett verteilen. Insgesamt, gibt es also maximal  $15^x$  Möglichkeiten, die  $x$  weißen Bauern und  $x$  schwarzen Bauern auf die  $x$  blockierten Spalten zu verteilen.

Nun verteilen wir die restlichen Bauern auf dem Schachbrett. Da es maximal 48 Felder gibt, auf den ein Bauer stehen kann und bereits  $2x$  Bauern positioniert wurden, gibt es noch maximal  $\binom{48-2x}{wB-x}$  Möglichkeiten die  $wB - x$  restlichen weißen Bauern zu verteilen. Für die restlichen  $sB - x$  schwarzen Bauern gibt es dann noch  $48 - wB - x$  verfügbare Felder, weshalb es maximal  $\binom{48-wB-x}{sB-x}$  Möglichkeiten gibt, diese auf dem Schachbrett zu verteilen. Den Faktor  $y$  kennen wir bereits aus Satz 3.8, nur dass hier die Gesamtanzahl der weißen bzw. schwarzen Bauern anstatt der Anzahl der weißen bzw. schwarzen Bauern in den Reihen vier und fünf verwendet wird. Diese Änderung macht  $y$  aber in keinen Fall kleiner als  $E_{ijkl}$ , weshalb der Beweis von  $y$  analog zum Beweis von  $E_{ijkl}$  in Satz 3.8 vollzogen werden kann. Somit berücksichtigt  $y$  die möglichen En-Passant-Situationen und das Zugrecht und schließlich folgt, dass  $M$  tatsächlich die obere Schranke ist.  $\square$



Abb. 10: Die Bauernblockaden (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (3, 4), (3, 5) und (3, 6)



Abb. 11: Die Bauernblockaden (3, 7), (4, 5), (4, 6), (4, 7), (5, 6), (5, 7) und (6, 7)

Nach den Bauern positioniert Chinchalkar die weißen Türme und den weißen König auf dem Schachbrett.

**Satz 3.10**  $W_{wK}$  ist eine obere Schranke an die Anzahl der Möglichkeiten den weißen König und die  $wT$  weißen Türme auf die  $S$  verfügbaren Felder des Schachbretts zu verteilen, die die möglichen Rochade-Situationen berücksichtigt, wobei

$$W_{wK} = \binom{S-1}{wT} \cdot S + \binom{S-1}{wT} \cdot f \text{ mit } f = \begin{cases} 1 & \text{für } wT = 0 \\ 3 & \text{für } wT = 1 \\ 4 & \text{für } wT \geq 2 \end{cases}$$

**Beweis** Der Beweis stammt aus [1] und wurde von mir etwas ausführlicher dargestellt. Es können bei der Platzierung des Königs zwei Fälle auftreten. Wir werden für jeden Fall eine obere Schranke finden und diese dann addieren.

Fall 1: Der König steht nicht auf e1: Eine Rochade ist unmöglich, da der König bewegt wurde. Zuerst platzieren wir den König, dafür gibt es nicht mehr als  $S$  Möglichkeiten. Die Anzahl an Möglichkeiten die  $wT$  Türme danach zu positionieren wird durch den Binomialkoeffizienten  $\binom{S-1}{wT}$  beschrieben. Damit ist  $\binom{S-1}{wT} \cdot S$  eine obere Schranke für die Anzahl der Möglichkeiten in diesem Fall.

Fall 2: Der König steht auf e1: Nach Voraussetzung gibt es genau eine Möglichkeit den König zu platzieren. Da der König noch auf seinem Startfeld steht, ist eine Rochade grundsätzlich nicht ausgeschlossen. Die Variable  $f$  beschreibt die Anzahl der verschiedenen Rochade-Situationen für Weiß. Gibt es keinen Turm ( $wT = 0$ ), ist nicht zu rochieren die einzige mögliche Rochade-Situation, deshalb ist  $f = 1$ . Gibt es einen Turm ( $wT = 1$ ), kann es zu  $f = 3$  Rochade-Situationen kommen. Es ist möglich, dass nur königsseitig, nur damenseitig oder gar nicht rochiert werden kann. Bei  $wT \geq 2$  kann es zu den im Fall  $r = 1$  genannten Rochade-Situationen kommen. Darüber hinaus gibt es noch die Möglichkeit, dass beidseitig rochiert werden kann, deshalb ist in diesem Fall  $f = 4$ . Für die Platzierung der Türme gibt es maximal  $\binom{S-1}{wT}$  Möglichkeiten. Damit ist  $\binom{S-1}{wT} \cdot f$  eine obere Schranke für diesen Fall.

Schließlich folgt, dass  $W_{wK} = \binom{S-1}{wT} \cdot S + \binom{S-1}{wT} \cdot f$  die besagte obere Schranke ist.  $\square$

Chinchalker erwähnt, dass eine obere Schranke für die Positionierung des schwarzen Königs und der schwarzen Türme analog gefunden werden kann, wir nennen diese Schranke  $W_{sK}$ .

Die restlichen Figurengruppen werden nun nacheinander auf dem Schachbrett positioniert. Die Anzahl an Möglichkeiten für diese Positionierung kann mit einem Multinomialkoeffizienten berechnet werden. Insgesamt wird demnach für eine Figurenkombination  $x = (wB, wD, wT, wL, wS, sB, sD, sT, sL, sS)$  die obere Schranke  $\Theta_x$  wie folgt berechnet:

$$\Theta_x = M \cdot W_{wK} \cdot W_{sK} \cdot \binom{62 - wB - sB - wT - sT}{wD, wL, wS, sD, sL, sS}$$



Durch das Aufsummieren der  $\Theta_x$  über alle Figurenkombinationen  $x \in X$  erhält Chinchalkar die obere Schranke  $O \approx 1,77894 \cdot 10^{46}$  an die Anzahl der legalen Schachpositionen.

Betrachten wir nun die Umsetzung der Resultate im Code von *npos ORIG.c*. Die Funktion `arrange` (Z. 123-194) nimmt eine Figurenkombination  $x$  als Input und gibt  $\Theta_x$  als Rückgabewert aus. Die obere Schranke  $W_{Bauer}$  aus Satz 3.8 wird für die verschiedenen Werte von  $wB$  und  $sB$  in der Funktion `place_pawns` (Z. 64-95) berechnet und im Array `pawns` gespeichert. Die obere Schranke  $M$  aus Satz 3.9 wird in den Zeilen 130 bis 145 berechnet. Die Funktion `place_rooks` (Z. 106-121) berechnet die obere Schranke  $W_{wK}$  aus Satz 3.10, indem sie die Anzahl der Türme (Variable `r`) und die Anzahl der verbleibenden freien Felder (Variable `l`) als Inputparameter verwendet. Auch die obere Schranke  $W_{sK}$  wird mit `place_rooks` berechnet. In der Sequenz in den Zeilen 160 bis 172 wird die Verteilung der restlichen Figuren mit Binomialkoeffizienten berechnet. Das Programm gibt die obere Schranke  $O$  mit  $1,77894 \cdot 10^{46}$  als Ergebnis aus.

## 4 Tromps Lösung

Im Jahr 2022 konnte Tromp die obere Schranke von Chinchalkar mit der Zahl  $8.726.713.169.886.222.032.347.729.969.256.422.370.854.716.254 \approx 8,73 \cdot 10^{45}$  unterbieten. Der Programmcode, mit dem er diese obere Schranke berechnet hat, ist in der Datei *CountChess.hs* seines Github-Posts [5] zu finden. Da bei mir Laufzeitprobleme aufgetreten sind, als ich Tromps Haskell-Code modifizieren wollte, habe ich seinen Programmcode in C übersetzt und meine Erweiterung auf Basis dieser Übersetzung (s. Anhang) geschrieben. Beim Übersetzen habe ich sichergestellt, dass alle Funktionswerte, Zwischenergebnisse und Endergebnisse des Originals und der Übersetzung übereinstimmen. Da es deutlich wahrscheinlicher ist, dass der Leser C-Code versteht, werden wir auf den C-Code verweisen. Quelle [5] ist die einzige Arbeit von Tromp, auf die sich meine Arbeit bezieht.

Tromp hat seine Lösung ebenfalls in zwei Schritte aufgeteilt. Zuerst filtert er möglichst viele illegale Figurenkombinationen heraus und findet dann für jede verbleibende Figurenkombination in Abhängigkeit von Rochade- und En-Passant-Situation eine obere Schranke an die Anzahl der Möglichkeiten diese Figuren auf dem Schachbrett zu verteilen. Abschließend summiert er alle oberen Schranken auf und multipliziert diese Summe mit einigen Multiplikatoren. Wir werden in diesem Kapitel die von Tromp verwendeten Konzepte als Sätze und Lemmata formulieren und beweisen.

Alle in diesem Kapitel verwendeten Bedingungen und Konzepte stammen aus dem Code von Tromp und wurden von ihm größtenteils durch Kommentare grundlegend erklärt. Jedoch überlässt er es dem Leser die Gültigkeit dieser Bedingungen zu überprüfen. Deshalb sind alle Beweise dieses Kapitels von mir.

### 4.1 Fixierte Figuren und Multiplikatoren

Bei der Umsetzung des Zugrechts, der En-Passant-Situation und der Rochade-Situationen weisen die Lösungen von Tromp und Chinchalkar große Unterschiede auf. Wir werden nun schrittweise diese Umsetzungen durchgehen und beginnen dabei mit dem Zugrecht.

Tromp geht bei jeder seiner Berechnungen davon aus, dass Weiß am Zug ist und multipliziert lediglich das Endergebnis mit zwei (Z. 272), um auch alle Schachpositionen zu berücksichtigen, bei denen Schwarz am Zug ist. Auch wir werden in den nächsten beiden Kapiteln davon ausgehen, dass Weiß am Zug ist. Alle Aussagen können analog bewiesen werden, wenn Schwarz am Zug ist.

Tromp führt zur Umsetzung der Rochade-Situationen die Begriffe fixierter Turm und fixierter König ein. Die Spielregeln besagen, dass eine Seite nur dann rochieren darf, wenn sowohl der König als auch der beteiligte Turm noch nicht gezogen haben. Daraus folgt, dass die vorliegende Rochade-Situation bestimmt, welche Türme noch nicht gezogen haben können. Türme, die aufgrund der vorliegenden Rochade-Situation noch nicht gezogen haben können, nennen wir fixierte Türme. Sobald eine Seite mindestens einen fixierten Turm hat, muss

der König auch auf seinem Anfangsfeld stehen, da sonst keine Rochade möglich wäre. Ein solcher König heißt fixierter König.

**Definition 4.1 (Fixierter Turm)** Ein Turm heißt fixiert, wenn die vorliegende Rochade-Situation verlangt, dass dieser Turm noch nicht gezogen haben kann. Die Anzahl an weißen fixierten Türmen ist  $fwr \in \{0, 1, 2\}$  und die Anzahl an schwarzen fixierten Türmen ist  $fbr \in \{0, 1, 2\}$ .

**Definition 4.2 (Fixierter König)** Ein König heißt fixierter König, wenn die Seite des Königs mindestens einen fixierten Turm hat. Die Anzahl der fixierten weißen Könige ist  $fixwk \in \{0, 1\}$  und die Anzahl der fixierten schwarzen Könige ist  $fixbk \in \{0, 1\}$ .

Das folgenden Lemma gilt analog für Schwarz mit  $fbr$ .

**Lemma 4.3** Weiß kann...

- a) nicht mehr rochieren  $\Leftrightarrow fwr = 0$
- b) nur noch damenseitig oder nur noch königsseitig rochieren  $\Leftrightarrow fwr = 1$
- c) beidseitig rochieren  $\Leftrightarrow fwr = 2$

**Beweis** a), b) und c) folgen direkt aus der Definition des fixierten Turms.  $\square$

Während es bei  $fwr \in \{0, 2\}$  nur eine Möglichkeit gibt, die fixierten weißen Türme zu platzieren, kann der fixierte weiße Turm bei  $fwr = 1$  sowohl auf dem Feld a1 (für eine damenseitige Rochade) als auch auf dem Feld h1 (für eine königsseitige Rochade) stehen. Der Multiplikator  $multFWR$  gibt die Anzahl an Möglichkeiten an die  $fwr$  weißen fixierten Türme zu platzieren. Es gilt:

$$multFWR = \begin{cases} 1 & \text{für } fwr \in \{0, 2\} \\ 2 & \text{für } fwr = 1 \end{cases}$$

Für Schwarz ist  $multFBR$  analog zu  $multFWR$  definiert.

Die En-Passant-Situation implementiert Tromp, indem er lediglich unterscheidet, ob Weiß in diesem Zug En-Passant schlagen kann oder nicht. Falls Weiß En-Passant schlagen kann, muss es sowohl einen weißen Bauern in der fünften Reihe geben, als auch einen schwarzen Bauern, der im Zug zuvor in die fünfte Reihe gezogen wurde. Da wir uns später mit der Anzahl an Möglichkeiten für die Positionierung der beiden Bauern beschäftigen werden, können wir momentan davon ausgehen, dass diese beiden Bauern jeweils in einer ganz bestimmten Spalte stehen und somit ihr jeweiliges Feld fest vorgegeben ist.

**Definition 4.4 (Fixierter Bauer)** Ein Bauer heißt fixiert, wenn die En-Passant-Situation verlangt, dass dieser Bauer auf einem bestimmten Feld steht. Die Anzahl an fixierten Bauern pro Seite ist  $fixp \in \{0, 1\}$ . Eine Spalte heißt fixierte Spalte, wenn sich in ihr ein fixierter Bauer befindet.

Außerdem verlangt die En-Passant-Situation, dass der schwarze fixierte Bauer im letzten Zug gezogen wurde.

Von nun an sei  $wp$  die Anzahl der nicht-fixierten weißen Bauern und  $bp$  die Anzahl der nicht-fixierten schwarzen Bauern. Das nachfolgende Lemma zeigt, dass der Wert von  $fixp$  die vorliegende En-Passant-Situation anzeigt.

**Lemma 4.5** Weiß kann...

- a) nicht En-Passant schlagen  $\Leftrightarrow fixp = 0$
- b) En-Passant schlagen  $\Leftrightarrow fixp = 1$

**Beweis** Wir werden a) beweisen, b) kann analog bewiesen werden.

$\Rightarrow$ : Kann Weiß nicht En-Passant schlagen, müssen keine Bauern auf bestimmten Feldern stehen, um ein En-Passant schlagen zu ermöglichen, deshalb gilt  $fixp = 0$ .

$\Leftarrow$ : Wenn  $fixp = 0$  gilt, verlangt die En-Passant-Situation nicht, dass Bauern auf bestimmten Feldern stehen. Falls En-Passant schlagen möglich wäre, müssten Bauern auf bestimmten Feldern stehen. Deshalb kann Weiß nicht En-Passant schlagen.  $\square$

Wie bereits in der Definition angedeutet wurde, ist die Anzahl der fixierten weißen Bauern immer gleich der Anzahl der fixierten schwarzen Bauern. Da dies nicht intuitiv klar ist, ist es wichtig zu verstehen, dass es einen Unterschied zwischen fixierten weißen Bauern und weißen Bauern, die En-Passant schlagen können, gibt. Es existieren Schachpositionen, in denen Weiß mit zwei verschiedenen Bauern En-Passant schlagen kann (s. Abb 12). Es ist aber immer nur ein weißer Bauer fixiert, da immer nur ein weißer Bauer für ein mögliches En-Passant-Schlagen auf einem bestimmten Feld stehen muss und die En-Passant-Situationen keinen zweiten weißen Bauern auf einem bestimmten Feld benötigt.



Abb. 12: Der letzte Zug von Schwarz war Bauer von d7 auf d5. Sowohl der weiße Bauer auf c5 als auch der weiße Bauer auf e5 kann den schwarzen Bauern auf d5 En-Passant schlagen.

Zuvor habe ich erwähnt, dass wir uns später darum kümmern werden, wie viele Möglichkeiten es gibt, die fixierten Bauern in der fünften Reihe zu platzieren. Diese Anzahl wird durch den Multiplikator *multEP* angegeben.

**Lemma 4.6** *Es gilt:*

$$multEP = \begin{cases} 1 & \text{für } fixp = 0 \\ 14 & \text{für } fixp = 1 \end{cases}$$

**Beweis** Wir werden nacheinander die Fälle  $fixp = 0$  und  $fixp = 1$  beweisen.

Fall  $fixp = 0$ : Es gibt genau eine Möglichkeit null fixierte Bauern auf die acht Spalten zu verteilen.

Fall  $fixp = 1$ : Steht der schwarze fixierte Bauer in den Spalten b, c, d, e, f oder g, muss der weiße fixierte Bauer entweder rechts oder links neben ihm stehen, um ihn En-Passant schlagen zu können. Steht der schwarze fixierte Bauer in der Spalte a, muss der weiße fixierte Bauer in der Spalte b stehen und steht der schwarze fixierte Bauer in der Spalte h, muss der weiße fixierte Bauer in der Spalte g stehen, damit er ihn En-Passant schlagen kann. Insgesamt gibt es somit maximal 14 Möglichkeiten.  $\square$

Da der schwarze fixierte Bauer der Bauer ist, der im letzten Zug gezogen wurde, ist es für Tromp nicht nötig - im Gegensatz zu Chinchalkar in Satz 3.8 - die Anzahl an En-Passant schlagbaren Bauern zu ermitteln, da der fixierte schwarze Bauer der einzige Bauer ist, der En-Passant geschlagen werden kann. Durch

den Multiplikator *multEP* werden unter anderem alle möglichen Positionen des schwarzen fixierten Bauerns mitgezählt.

## 4.2 Umsetzung der Figurenkombinationen

Tromps Code beginnt damit, dass er mit Hilfe von Schleifen alle möglichen Wertekombinationen von *fwr*, *fbr* und *fixp* durchgeht. In der innersten Schleife wird die count-Funktion aufgerufen (Z. 264), die in Abhängigkeit von *fwr*, *fbr* und *fixp* eine obere Schranke an die Anzahl der legalen Schachpositionen berechnet. Die Multiplikatoren *multFWR*, *multFBR* und *multEP* werden in den Zeilen 252, 256 und 261 definiert und in Zeile 264 an das Ergebnis der count-Funktion heran multipliziert.

Tromps Berechnung von möglichen Figurenkombinationen unterscheidet sich durch die Einführung von fixierten Bauern, Türmen und Königen in einigen Aspekten von Chinchalkars Berechnung. Da bereits die Anzahl der fixierten Bauern und Türme als Inputparameter der count-Funktion feststehen, verwendet Tromp die Anzahl der nicht-fixierten Bauern und Türme. Außerdem berechnet Tromp auch die Anzahl der fixierten Könige pro Seite, da diese helfen die Anzahl der noch verfügbaren Felder zu verringern. Da wir die Variablen *wp* und *bp* im Verlauf dieser Arbeit noch öfter benötigen werden, definieren wir sie nun. Sei *wp* die Anzahl an nicht-fixierten weißen Bauern und *bp* die Anzahl an nicht-fixierten schwarzen Bauern.

Auch Tromp filtert einige illegale Figurenkombinationen heraus. Bei ihm geschieht das, indem er bereits in den Schleifenbedingungen die Anzahl der noch für Promotionen verfügbaren Bauern mit Hilfe von Hilfsvariablen wie *wnp1* einbezieht und somit nicht am Ende prüfen muss, ob zu viele Promotionen stattgefunden haben. Als weitere Filter verwendet er die folgenden beiden Bedingungen.

**Lemma 4.7** *Es gilt:*

$$a) \quad bpx + caps - wproms \geq 0$$

$$b) \quad wpx + caps - bproms \geq 0$$

**Beweis** Wir beweisen a), b) kann analog bewiesen werden. Stellt man a) um, ergibt sich  $bpx + caps \geq wproms$ . Sei *gx* die Anzahl an Captures, in denen kein schwarzer Bauer geschlagen wurde, dann gilt  $caps = gx + bpx$  und es folgt  $2 \cdot bpx + gx \geq wproms$ . Nach Satz 3.5 gilt, dass für jeden geschlagenen schwarzen Bauern maximal zwei weiße Promotionen durchführbar sind und für jede andere geschlagene Figur maximal eine. Demnach kann Weiß maximal  $2 \cdot bpx + gx$  Promotionen durchführen und somit muss  $bpx + caps - wproms \geq 0$  gelten.  $\square$

Anders als Chinchalkar berechnet Tromp nie die exakte Anzahl an Figurenkombinationen. Ein Vergleich der Anzahlen hätte auch wenig Aussagekraft, da es bei Tromp durch die verschiedenen Werte von  $fixp$ ,  $fwr$  und  $fbr$  deutlich mehr Figurenkombinationen gäbe.

### 4.3 Berechnung der Bauernverteilung

In diesem Unterkapitel betrachten wir die Berechnung der Anzahl an Möglichkeiten eine Figurenkombination in Abhängigkeit von  $fixp$ ,  $fwr$  und  $fbr$  auf dem Schachbrett zu verteilen. Da fixierte Figuren auf bestimmten Feldern stehen müssen, werden sie zuerst auf das Schachbrett gestellt. Als zweites werden die Bauern platziert. Die Berechnung einer oberen Schranke an die Anzahl an möglichen Bauernverteilungen stellt den größten Unterschied zwischen den Lösungen von Tromp und Chinchalkar dar. Wir beginnen die Erklärung von Tromps Berechnung, indem wir die Variablen  $maxuwp$  und  $minopp$  einführen.

**Lemma 4.8** *Seien die beiden Variablen  $maxuwp$  und  $minopp$  wie folgt definiert:*

- $maxuwp := bpx + caps - wproms$
- $minopp := \max(0, fixp + wp - maxuwp)$

*Dann gilt:*

- a)  $maxuwp$  ist die Maximalanzahl an weißen Bauern, die in freien Spalten stehen
- b)  $minopp$  ist die Mindestanzahl an blockierten Spalten

**Beweis** Wir beweisen zuerst a) und benutzen dann a) um b) zu beweisen.

Beweis a): Laut dem Beweis von Lemma 4.7 ist  $bpx + caps$  die Maximalanzahl an weißen Promotionen. Da  $wproms$  die tatsächliche Anzahl an weißen Promotionen ist, ist  $bpx + caps - wproms$  die Maximalanzahl der noch nicht durchgeführten aber ohne weiteres Capture möglichen weißen Promotionen. Genau die Bauern, die in freien Spalten stehen, können eine Promotion ohne weiteres Capture durchführen. Deshalb gilt a).

Beweis (b): Wenn maximal  $maxuwp$  weiße Bauern in freien Spalten stehen, müssen mindestens  $\max(0, fixp + wp - maxuwp) = minopp$  weiße Bauern in blockierten Spalten stehen. Wir behaupten, dass diese  $minopp$  weißen Bauern in nicht weniger als  $minopp$  Spalten stehen können, woraus folgt, dass  $minopp$  die Mindestanzahl an blockierten Spalten ist.

Wir beweisen die Behauptung mithilfe eines Widerspruchsbeweises. Angenommen es gibt genau  $y$  weiße Bauern, die in  $y - z$  blockierten Spalten stehen, wobei  $y - z < minopp$  und  $y \geq minopp$  ist. Da ohne Spaltenwechsel maximal

$y - z$  weiße Bauern in den  $y - z$  blockierten Spalten stehen würden, muss es mindestens  $z$  Spaltenwechsel weißer Bauern und somit auch mindestens  $z$  Captures gegeben haben, bei denen ein weißer Bauer eine Figur, die in einer dieser  $y - z$  blockierten Spalten stand, geschlagen hat. Wenn ein weißer Bauer eine Figur schlägt, die in einer blockierten Spalte steht, ist durch das Capture mindestens eine weiße Promotion weniger möglich, als durch ein Capture mit einer freien (oder frei werdenden) Spalte als Opfer-Spalte möglich gewesen wäre. Dies ist der Fall, weil der schlagende weiße Bauer in einer blockierten Spalte keine Promotion ohne weiteres Capture durchführen kann, in einer freien (oder frei werdenden) Spalte jedoch schon. Daraus folgt, dass es in dieser Situation nur maximal  $bpx + caps - z$  weiße Promotionen geben kann. Demnach kann es maximal  $bpx + caps - wproms - z$  weiße Bauern in freien Spalten geben und somit gibt es mindestens  $minopp + z$  weiße Bauern in blockierten Spalten. Da  $minopp + z > y$  ist, widerspricht dies der Annahme, dass es genau  $y$  weiße Bauern in weniger als  $minopp$  blockierten Spalten gibt. Da  $y \geq minopp$  gilt, ist es nicht möglich, dass mindestens  $minopp$  weiße Bauern in weniger als  $minopp$  blockierten Spalten stehen.  $\square$

Im Code werden die Variablen *maxuwp* und *minopp* in den Zeilen 210 und 218 berechnet. Die wichtigere der beiden Variablen ist *minopp*, da wir von nun an in jeder Situation annehmen können, dass es mindestens *minopp* blockierte Spalten gibt. Diese Annahme wird uns helfen die Anzahl an möglichen Bauernverteilungen deutlich einzuschränken. Ein weiteres hilfreiches Konzept lernen wir nun mit der Definition des eingeschlossenen Feldes kennen.

**Definition 4.9 (Eingeschlossenes Feld)** Ein Feld  $ij$  heißt eingeschlossen, wenn es Felder  $iw$  und  $is$  mit  $i \in \{a, \dots, h\}$  und  $j, w, s \in \{1, \dots, 8\}$  gibt, sodass folgende Bedingungen gelten:

- $w < j < s$
- auf  $iw$  befindet sich ein weißer und auf  $is$  ein schwarzer Bauer
- alle Felder  $ik$  mit  $k \in \{w + 1, \dots, s - 1\}$  sind nicht von Bauern besetzt

Man sagt auch, dass Bauern das Feld  $ij$  einschließen.

Es können nur Felder in blockierten Spalten eingeschlossen werden, da sonst der erste Stichpunkt der Definition nicht erfüllt ist.





Abb. 13: In dieser Abbildung wird das Konzept der eingeschlossenen Felder anhand von Beispielen veranschaulicht.

Spalte a: Es gibt keinen schwarzen Bauern, der dem weißen Bauern gegenüber steht, deshalb kann es keine eingeschlossenen Felder in dieser Spalte geben.

Spalte b: Es gibt zwar weiße und schwarze Bauern, die sich gegenüberstehen, jedoch sind alle Felder zwischen ihnen mit Bauern besetzt, sodass es auch in dieser Spalte keine eingeschlossenen Felder gibt.

Spalte c: Es stehen zwar keine Bauern zwischen dem schwarzen Bauern und dem weißen Bauern, jedoch steht der weiße Bauer in einer höheren Reihe als der schwarze Bauer, weshalb es auch hier keine eingeschlossenen Felder gibt.

Spalte d: Erstmals sind alle Bedingungen erfüllt und das Feld d4 ist ein eingeschlossenes Feld.

Spalte e: Auch das Feld e4 ist ein eingeschlossenes Feld, da ein Turm kein Bauer ist und somit akzeptiert wird. Andere Figuren werden im Bauernterritorium akzeptiert, da diese erst später auf das Schachbrett gestellt werden und somit die Bauernverteilung nicht behindern.

Spalte f: Lediglich das Feld f5 ist eingeschlossen, da der Bauer auf f4 verhindert, dass alle Felder zwischen den Bauern auf f2 und f6 nicht von Bauern besetzt sind.

Nachdem wir nun ein gutes Verständnis von eingeschlossenen Feldern haben, können wir uns dem folgenden Satz widmen.

**Satz 4.10** Sei  $Z_{p,s,n}$  die Anzahl an Möglichkeiten  $s$  Felder von Bauern in  $p$  blockierten Spalten einzuschließen und diese  $p$  blockierten Spalten aus  $n$  verfügbaren Spalten auszuwählen. Sei  $p \leq n$ . Es gilt  $Z_{p,s,n} = m(p, s, n)$ , wobei

$$m(p, s, n) = \begin{cases} y & \text{für } p > 0 \\ 1 & \text{für } p = s = 0 \\ 0 & \text{für } p = 0 \wedge s > 0 \end{cases}$$

$$\text{mit } y = m(p, s, n-1) + \sum_{i=0}^{\min(s,4)} (5-i) \cdot m(p-1, s-i, n-1)$$

**Beweis** Wir beweisen die Behauptung mit vollständiger Induktion über die Anzahl der verfügbaren Spalten  $n$ .

Induktionsanfang:  $n = 0$

Fall  $p = s = 0$ :

Es gilt  $Z_{0,0,0} = 1$ , da es nur eine Möglichkeit gibt, null Felder in null blockierten Spalten von Bauern einzuschließen und diese null blockierten Spalten aus null verfügbaren auszuwählen. Damit ist  $Z_{0,0,0} = m(0, 0, 0)$ , da nach Definition  $m(0, 0, 0) = 1$  gilt.

Fall  $p > 0$ :

Dieser Fall tritt wegen  $p \leq n$  bei  $n = 0$  nicht ein.

Fall  $p = 0 \wedge s > 0$ :

Nach Definition gilt  $m(0, s, 0) = 0$ . Darüber hinaus gilt  $Z_{0,s,0} = 0$ , da es nicht möglich ist mehr als null von Bauern eingeschlossene Felder auf null blockierte Spalten zu verteilen.

Induktionsvoraussetzung: Für ein festes, aber beliebiges  $n \in \mathbb{N}$ , gilt, dass für alle  $k \in \{0, \dots, n\}$  gilt, dass die Aussage aus Satz 4.10 für  $k$  verfügbare Spalten gilt.

Induktionsschritt:  $n \Rightarrow n + 1$

Sei  $X_{p,s}$  die Anzahl an Möglichkeiten  $s$  Felder in  $p$  blockierten Spalten von Bauern einzuschließen, dann gilt für alle  $n$ :

$$Z_{p,s,n} = X_{p,s} \cdot \binom{n}{p} \quad (1)$$

Man multipliziert  $X_{p,s}$  mit der Anzahl an Möglichkeiten die  $p$  blockierten Spalten aus den  $n$  verfügbaren Spalten auszuwählen, um  $Z_{p,s,n}$  zu erhalten. Daraus folgt:

$$Z_{p,s,n+1} = \frac{\binom{n+1}{p}}{\binom{n}{p}} \cdot Z_{p,s,n} = \frac{n+1}{n-p+1} \cdot Z_{p,s,n} \quad (2)$$

Da  $p \leq n$  gilt, ist der Bruch für alle Werte von  $p$  und  $n$  definiert. Diese Gleichung wird uns beim Beweisen der verschiedenen Fälle nützlich sein.

Fall  $p = s = 0$ :

Es gilt definitionsgemäß  $m(0, 0, n+1) = 1$ . Wegen der Induktionsvoraussetzung gilt  $Z_{0,0,n} = 1$  und da  $p = 0$  ist, folgt mit (2), dass auch  $Z_{p,s,n+1} = 1$  gilt.

Fall  $p = 0 \wedge s > 0$ :

Nach Definition gilt  $m(p, s, n) = m(p, s, n+1) = 0$  und damit folgt durch die Induktionsvoraussetzung, dass  $0 = m(p, s, n) = Z_{p,s,n}$  ist. Wegen (2) sehen wir dann, dass auch  $Z_{p,s,n+1} = 0$  ist.

Fall  $p > 0$ :

Folgendes ist zu zeigen:

$$Z_{p,s,n+1} = m(p, s, n) + \underbrace{\sum_{i=0}^{\min(s,4)} (5-i) \cdot m(p-1, s-i, n)}_{=:A}$$

Da nach Induktionsvoraussetzung  $Z_{p,s,n} = m(p, s, n)$  gilt, folgt mit (2):

$$m(p, s, n) = \frac{n-p+1}{n+1} \cdot Z_{p,s,n+1}$$

Nach Induktionsvoraussetzung gilt außerdem  $m(p-1, s-i, n) = Z_{p-1,s-i,n}$  für alle  $i \in \{0, \dots, s\}$  und deshalb können wir  $A$  auch schreiben als:

$$A = \sum_{i=0}^{\min(s,4)} (5-i) \cdot Z_{p-1,s-i,n} = \binom{n}{p-1} \cdot \sum_{i=0}^{\min(s,4)} (5-i) \cdot X_{p-1,s-i}$$

Betrachten wir nun die Summe genauer.  $(5-i)$  gibt an, wie viele Möglichkeiten es gibt  $i$  Felder in einer blockierten Spalte von Bauern einzuschließen (s. Abb. 14-16). Daraus folgt, dass  $(5-i) \cdot X_{p-1,s-i}$  angibt, wie viele Möglichkeiten es gibt  $i$  Felder in einer blockierten Spalte von Bauern einzuschließen und  $s-i$  weitere Felder in  $p-1$  weiteren blockierten Spalten von Bauern einzuschließen. Da über jede mögliche Anzahl an Feldern, die in einer blockierten Spalte von Bauern eingeschlossen werden können, summiert wird, gilt:

$$X_{p,s} = \sum_{i=0}^{\min(s,4)} (5-i) \cdot m(p-1, s-i, n)$$

Weil Gleichung (1) für alle  $n$  gilt, gilt sie auch für  $Z_{p,s,n+1}$ . Durch Umstellen erhalten wir:

$$X_{p,s} = \frac{Z_{p,s,n+1}}{\binom{n+1}{p}}$$

Deshalb können wir für  $A$  folgern:

$$A = \frac{\binom{n}{p-1}}{\binom{n+1}{p}} \cdot Z_{p,s,n+1} = \frac{p}{n+1} \cdot Z_{p,s,n+1}$$

Fügen wir unsere Ergebnisse zusammen, folgt schließlich:

$$m(p, s, n+1) = m(p, s, n) + A = \left( \frac{n-p+1}{n+1} + \frac{p}{n+1} \right) \cdot Z_{p,s,n+1} = Z_{p,s,n+1}$$

□

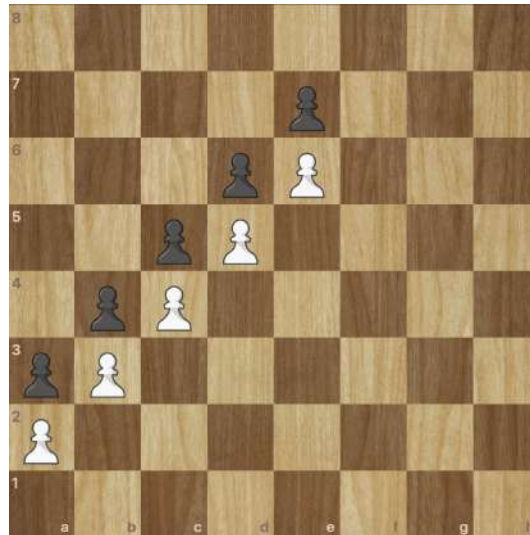


Abb. 14: Dies sind alle Möglichkeiten null Felder in einer Spalte von Bauern einzuschließen.



Abb. 15: Links sind alle Möglichkeiten ein Feld und rechts alle Möglichkeiten zwei Felder in einer Spalte von Bauern einzuschließen.

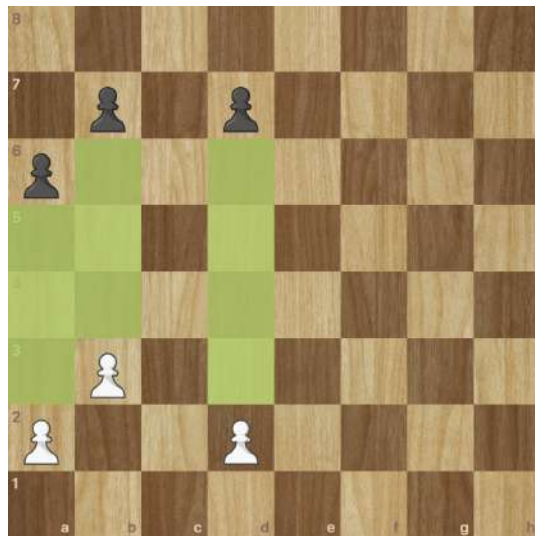


Abb. 16: Links sind alle Möglichkeiten drei Felder und rechts alle Möglichkeiten vier Felder in einer Spalte von Bauern einzuschließen.

Im Code werden die Werte von  $m(p, s, n)$  mit den Funktionen `opps` und `mopps-function` berechnet und im Array `mopps` gespeichert (Z. 45-70). Mithilfe von Satz 4.10 können wir nun eine obere Schranke an die Anzahl an möglichen Bauernverteilungen bestimmen, wenn En-Passant schlagen nicht möglich ist.

**Satz 4.11**  $B_{0,wp,bp}$  ist eine obere Schranke an die Anzahl an Möglichkeiten  $wp$  nicht-fixierte weiße Bauern und  $bp$  nicht-fixierte schwarze Bauern auf dem Schachfeld zu platzieren, wenn nicht En-Passant geschlagen werden kann, wobei

$$B_{0,wp,bp} = \sum_{s=0}^{4 \cdot \text{minopp}} m(\text{minopp}, s, 8) \cdot \binom{48 - 2 \cdot \text{minopp} - s}{wp - \text{minopp}, bp - \text{minopp}}$$

**Beweis** Laut Lemma 4.8 hat jede mögliche Bauernverteilung nicht weniger als  $\text{minopp}$  blockierte Spalten, demnach können wir  $\text{minopp}$  in  $m(p, s, n)$  einsetzen. Jeder Summand berechnet die Anzahl an Möglichkeiten die  $wp$  nicht-fixierten weißen Bauern und die  $bp$  nicht-fixierten schwarzen Bauern auf dem Feld zu verteilen, unter der Bedingung, dass genau  $s$  Felder von Bauern in den  $\text{minopp}$  blockierten Spalten eingeschlossen werden.  $m(p, s, n)$  berechnet dabei die Anzahl an Möglichkeiten die  $\text{minopp}$  weißen Bauern und  $\text{minopp}$  schwarzen Bauern, die in den blockierten Spalten stehen müssen, zu verteilen und der Multinomialkoeffizient die Anzahl an Möglichkeiten die restlichen Bauern zu verteilen. Es können in jeder blockierten Spalte zwischen null und vier Felder von Bauern eingeschlossen werden, sodass die Maximalanzahl an eingeschlossenen Feldern in  $\text{minopp}$  blockierten Spalten  $4 \cdot \text{minopp}$  ist. Indem man über alle möglichen Werte von  $s$  summiert, erhält man eine obere Schranke an die Anzahl an Möglichkeiten  $wp$  nicht-fixierte weiße und  $bp$  nicht-fixierte schwarze Bauern auf dem Schachfeld zu platzieren, wenn nicht En-Passant geschlagen werden kann.  $\square$

**Bemerkung** Satz 4.11 inkludiert auch alle Bauernverteilungen mit mehr als  $\text{minopp}$  blockierten Spalten, da die restlichen Bauern so positioniert werden können, dass sie sich in weiteren Spalten blockieren. Ein "restlicher Bauer" kann darüber hinaus auch in einer blockierten Spalte stehen, aber eben nicht auf einem eingeschlossenen Feld.

Die obere Schranke aus Satz 4.11 wird im Code im ersten Teil der Funktion `pawnsfunction` (Z. 96-110) berechnet und im Array `pawns` gespeichert.

Betrachten wir nun den Fall, dass En-Passant schlagen möglich ist, also  $\text{fixp} = 1$  gilt. Wir nennen von nun an die Spalte mit dem weißen fixierten Bauern weiße fixierte Spalte und die Spalte mit dem schwarzen fixierten Bauern schwarze fixierte Spalte. Da die fixierten Spalten bereits die fixierten Bauern beinhalten, können sie bei einer Berechnung mit  $m(p, s, n)$  nicht als "normale" blockierte oder freie Spalten behandelt werden und müssen gesondert betrachtet werden. Deshalb gehen wir nun alle Fälle durch, die in diesen Spalten auftreten können. Dabei gibt die Variable  $dw \in \{0, 1\}$  an, ob die schwarze fixierte Spalte blockiert

( $dw = 1$ ) oder frei ist ( $dw = 0$ ). Analog gibt dies  $db \in \{0, 1\}$  für die weiße fixierte Spalte an. Darüber hinaus beschreibt die Variable  $ds1 \in \{0, 1, 2\}$  wie viele Felder in der schwarzen fixierten Spalte von Bauern eingeschlossen werden und die Variable  $ds2 \in \{0, 1\}$  wie viele Felder in der weißen fixierten Spalte von Bauern eingeschlossen werden. Zum besseren Verständnis dienen die Abbildungen auf den nächsten Seiten.

Für die schwarze fixierte Spalte sind folgende Fälle ( $dw$ ,  $ds1$ ) möglich:

- Fall (0, 0): Freie Spalte
- Fall (1, 0): Blockierte Spalte, es gibt kein eingeschlossenes Feld
- Fall (1, 1): Blockierte Spalte, es gibt ein eingeschlossenes Feld
- Fall (1, 2): Blockierte Spalte, es gibt zwei eingeschlossene Felder

Für die weiße fixierte Spalte sind folgende Fälle ( $db$ ,  $ds2$ ) möglich:

- Fall (0, 0): Freie Spalte
- Fall (1, 0): Blockierte Spalte, es gibt kein eingeschlossenes Feld
- Fall (1, 1): Blockierte Spalte, es gibt ein eingeschlossenes Feld

Sei  $\Pi_w = \{(0, 0), (1, 0), (1, 1)\}$  und  $\Pi_s = \{(0, 0), (1, 0), (1, 1), (1, 2)\}$ . Die Fälle der beiden Spalten können beliebig kombiniert werden, sodass  $\Pi$  die Menge aller Kombinationen ist, mit:

$$\Pi = \{(dw, db, ds1, ds2) \mid (dw, ds1) \in \Pi_s, (db, ds2) \in \Pi_w\}$$



Abb. 17: Die schwarze fixierte Spalte ist frei. ( $dw = ds1 = 0$ )



Abb. 18: Die schwarze fixierte Spalte ist blockiert. Es wird kein Feld von Bauern eingeschlossen. ( $dw = 1$ ,  $ds1 = 0$ )



Abb. 19: Die schwarze fixierte Spalte ist blockiert. Es wird ein Feld von Bauern eingeschlossen. ( $dw = ds1 = 1$ )





Abb. 20: Die schwarze fixierte Spalte ist blockiert. Es werden zwei Felder von Bauern eingeschlossen. ( $dw = 1$ ,  $ds1 = 2$ )



Abb. 21: Die weiße fixierte Spalte ist frei. ( $db = ds2 = 0$ )



Abb. 22: Die weiße fixierte Spalte ist blockiert. Es wird kein Feld von Bauern eingeschlossen. ( $db = 1, ds2 = 0$ )



Abb. 23: Die weiße fixierte Spalte ist blockiert. Es wird ein Feld von Bauern eingeschlossen. ( $db = 1, ds2 = 1$ )

Mit diesem Wissen können wir nun auch eine obere Schranke an die Anzahl an möglichen Bauernverteilungen bestimmen, wenn  $fixp = 1$  gilt.

**Satz 4.12** Sei

- $\Pi$  definiert wie zuvor beschrieben
- $gopp = minopp - dw - db$
- $x = (dw, ds1, db, ds2)$

Dann ist  $B_{1,wp,bp}$  eine obere Schranke an die Anzahl an Möglichkeiten  $wp$  nicht-fixierte weiße und  $bp$  nicht-fixierte schwarze Bauern auf dem Schachfeld zu platzieren, wenn En-Passant geschlagen werden kann, mit

$$B_{1,wp,bp} = \sum_{x \in \Pi} \sum_{s=0}^{4 \cdot gopp} m(gopp, s, 6) \cdot \binom{44 - minopp - gopp - ds1 - ds2 - s}{wp - minopp + db, bp - minopp + dw}$$

**Beweis** Die Variable  $x$  wird lediglich als Platzhalter für  $(dw, ds1, db, ds2)$  verwendet, damit die Formel in eine Zeile passt.

$$\sum_{s=0}^{4 \cdot gopp} m(gopp, s, 6) \cdot \binom{44 - minopp - gopp - ds1 - ds2 - s}{wp - minopp + db, bp - minopp + dw}$$

beschreibt - ähnlich zu Satz 4.11 - die Anzahl an Möglichkeiten die nicht-fixierten Bauern auf dem Spielfeld zu verteilen. Im Vergleich zu Satz 4.11 müssen hier aufgrund der Existenz der fixierten Bauern einige Anpassungen vorgenommen werden.

Wie bereits zuvor erwähnt, werden die beiden fixierten Spalten nicht als "normale" blockierte Spalten behandelt und deshalb ist die Mindestanzahl an blockierten Spalten nur noch  $gopp := minopp - dw - db$ . Außerdem gibt es nur noch sechs Spalten (nämlich alle nicht-fixierten Spalten), aus denen die blockierten Spalten ausgewählt werden können. Aufgrund dessen wird hier  $m(gopp, s, 6)$  statt  $m(minopp, s, 8)$  verwendet und nur bis  $s = 4 \cdot gopp$  statt bis  $s = 4 \cdot minopp$  aufsummiert.

Der Multinomialkoeffizient beschreibt die Anzahl an Möglichkeiten die restlichen nicht-fixierten Bauern zu verteilen. Wir werden nun erklären, wie es zu den Werten  $44 - minopp - gopp - ds1 - ds2 - s$ ,  $wp - minopp + db$  und  $bp - minopp + dw$  im Multinomialkoeffizienten kommt.

Es wurden bereits  $minopp - db$  der  $wp$  nicht-fixierten weißen Bauern verteilt, sodass der Multinomialkoeffizient noch  $wp - minopp + db$  nicht-fixierte weiße Bauern verteilen muss. Analog müssen noch  $bp - minopp + dw$  nicht-fixierte schwarze Bauern verteilt werden.

$z := 2 + 2 + minopp + gopp + ds1 + ds2 + s$  Felder sind nicht mehr verfügbar.  $2 \cdot gopp$  Felder wurden bereits von den Bauern in den blockierten Spalten besetzt

und  $dw + db$  weitere Felder von nicht-fixierten Bauern in den fixierten Spalten. Es sei erwähnt, dass  $dw + db + 2 \cdot gopp = gopp + minopp$  ist. Sei die Spalte  $i \in \{a, \dots, h\}$  die schwarze fixierte Spalte, dann sind die Felder  $i6$  und  $i7$  nicht verfügbar, da der schwarze fixierte Bauer im letzten Zug von  $i7$  nach  $i5$  gezogen wurde und keine Figuren dabei im Weg stehen dürfen. Außerdem stehen auf zwei weiteren Feldern die beiden fixierten Bauern. Damit sind nur noch  $48 - z = 44 - minopp - gopp - ds1 - ds2 - s$  Felder für die Positionierung der restlichen Bauern verfügbar.

Weil über alle möglichen Situationen, die in den fixierten Spalten auftreten können, summiert wird, ist  $B_{1,wp,bp}$  eine obere Schranke an die Anzahl an Möglichkeiten  $wp$  nicht-fixierte weiße Bauern und  $bp$  nicht-fixierte schwarze Bauern auf dem Schachfeld zu platzieren, wenn En-Passant geschlagen werden kann.  $\square$

Im Code werden diese oberen Schranken im zweiten Teil der Funktion `pawnsfunction` berechnet (Z.114-138), indem für jede mögliche Kombination von  $dw$ ,  $db$ ,  $ds1$  und  $ds2$  das Ergebnis der Funktion `pawnsep` aufsummiert wird. Die Funktion `pawnsep` berechnet folgende Summe:

$$\sum_{s=0}^{4 \cdot gopp} m(gopp, s, 6) \cdot \binom{44 - minopp - gopp - ds1 - ds2 - s}{wp - minopp + db, bp - minopp + dw}$$

Nach der Platzierung der Bauern bleiben noch  $vF := 64 - 4 \cdot fixp - fixwk - fixbk - fwr - fbr - wp - bp$  Felder übrig (Z. 219 im Code), um die restlichen Figuren zu verteilen. Der Faktor  $4 \cdot fixp$  wird verwendet, weil vier Felder durch die Existenz der fixierten Bauern nicht besetzt werden können, nämlich die Felder auf denen die fixierten Bauern stehen, sowie die beiden Felder, die der schwarze fixierte Bauer im letzten Zug überquert hat. Die Verteilung der restlichen Figuren wird mit einem Multinomialkoeffizienten berechnet (Z. 221 im Code). Hierfür werden die in den Zeilen 174 und 197 berechneten Produkte von Fakultäten  $wprod$  und  $bprod$  verwendet um Laufzeit zu sparen.

Tromp erhält eine obere Schranke mit der Größe von ungefähr  $8,7 \cdot 10^{45}$  als Ergebnis seiner Berechnungen. Dies stellt eine Verbesserung von 50,9 % gegenüber der oberen Schranke von Chinchalkar dar. Die einzelnen Teilergebnisse für die verschiedenen Werte von  $fwr$ ,  $fbr$  und  $fixp$  sind in der folgenden Abbildung zu sehen. Es ist zu beachten, dass diese Werte noch nicht mit zwei multipliziert wurden und somit nur die Anzahl angeben, wenn Weiß am Zug ist.

```

[Running] cd "/Users/christianbuchs/Documents/Studium/Bachelorarbeit/C Project/Folder/" && gcc direktekopie
Stadium/Bachelorarbeit/C Project/Folder/"direktekopie
fwr = 0 fbr = 0 fixp = 0 gerundet: 4.317117e+45 exakt: 4317116501822869085736524809553156518216466432
fwr = 0 fbr = 0 fixp = 1 gerundet: 3.199960e+43 exakt: 31999595200554360726978853532556683563237376
fwr = 0 fbr = 1 fixp = 0 gerundet: 6.922143e+42 exakt: 6922142764258201498990574886765035111055360
fwr = 0 fbr = 1 fixp = 1 gerundet: 5.444438e+40 exakt: 54444384271358764337920637881472010682368
fwr = 0 fbr = 2 fixp = 0 gerundet: 1.365308e+41 exakt: 136530769983397185483934838882727423377408
fwr = 0 fbr = 2 fixp = 1 gerundet: 1.035366e+39 exakt: 1035365889137246104468971868036265934848
fwr = 1 fbr = 0 fixp = 0 gerundet: 6.922143e+42 exakt: 6922142764256625601320564597675088525918208
fwr = 1 fbr = 0 fixp = 1 gerundet: 5.430835e+40 exakt: 54308353406843120725028311050169928908800
fwr = 1 fbr = 1 fixp = 0 gerundet: 1.174542e+40 exakt: 11745419798220500118058263406702695022592
fwr = 1 fbr = 1 fixp = 1 gerundet: 9.817252e+37 exakt: 98172517157462428199580525005092421632
fwr = 1 fbr = 2 fixp = 0 gerundet: 2.359583e+38 exakt: 235958281120053141339249377081458425856
fwr = 1 fbr = 2 fixp = 1 gerundet: 1.903337e+36 exakt: 1903336650820530413600515787306565632
fwr = 2 fbr = 0 fixp = 0 gerundet: 1.365308e+41 exakt: 136530769983304939608194964218180643323904
fwr = 2 fbr = 0 fixp = 1 gerundet: 1.028638e+39 exakt: 1028637537647016138320248178990487961600
fwr = 2 fbr = 1 fixp = 0 gerundet: 2.359583e+38 exakt: 235958281119921255008115793629930323968
fwr = 2 fbr = 1 fixp = 1 gerundet: 1.895643e+36 exakt: 1895642836533361711037586679714021376
fwr = 2 fbr = 2 fixp = 0 gerundet: 4.729971e+36 exakt: 4729971278279345018382308284936749056
fwr = 2 fbr = 2 fixp = 1 gerundet: 3.663529e+34 exakt: 36635290891907343302044348485776384
Totale Summe: 8726713169814947753088220017782443029005598720

[Done] exited with code=1 in 9.248 seconds

```

Abb. 24: Ausgabe der C-Übersetzung von Tromps Programm

## 5 Meine Erweiterung

Meine Erweiterung basiert auf der C-Übersetzung des Codes von Tromp. Es müssen daher lediglich die Modifikationen, die von mir am Programm vorgenommen worden sind, bewiesen werden.

Eine Schachposition ist illegal, wenn eine Seite im Schach steht, jedoch nicht am Zug ist. Wir werden in meiner Erweiterung einige dieser illegalen Schachpositionen herausfiltern. Auch wir gehen davon aus, dass Weiß am Zug ist. Das heißt, dass jede Schachposition, in der der schwarze König im Schach steht, illegal ist. Um allgemein zu bestimmen, ob der schwarze König im Schach steht, benötigen wir folgende Informationen.

- wo der Schwarze König steht
- wo die Schach stellende Figur steht
- welches die Figur ist (denn ein Springer stellt anders Schach als ein Läufer)
- ob es Figuren gibt, die dieses Schach blockieren

Es erfordert eine sehr große Anzahl an Fallunterscheidungen, um all diese Informationen zu erhalten. Da dies den Rahmen der Bachelorarbeit sprengen würde, werde ich in diesem Kapitel lediglich einen spezifischeren Fall betrachten, der deutlich weniger Fallunterscheidungen benötigt. Wie ein Programm umgesetzt werden könnte, dass deutlich mehr illegale Schachpositionen herausfiltert, wird jedoch am Ende dieses Kapitels skizziert. Alle Teile dieses Kapitels sind meine Eigenleistung.

Da die Verbesserung von Tromps Schranke gegenüber der von Chinchalkar vor allem auf der intelligenten Berechnung der Bauernverteilungen beruht und wir diesen Vorteil nutzen möchten, werden auch wir zuerst die Bauern auf das Schachbrett stellen und erst dann den schwarzen König positionieren. Da die Funktionen von Tromp nur berechnen wie viele Bauernverteilungen es gibt, wissen wir nicht, wo die Bauern auf dem Schachbrett stehen, wenn wir den schwarzen König positionieren. Um dies herauszufinden, bräuchten wir erneut sehr viele Fallunterscheidungen und deshalb konzentrieren wir uns größtenteils auf die Felder, die nicht zum Bauernterritorium gehören.

Wir unterscheiden vier verschiedene Arten von Positionen des schwarzen Königs und nennen diese  $kpos \in \{0, \dots, 3\}$ .

- Fall  $kpos = 0$ : Er steht in einer der vier Ecken (a1, h1, a8 und h8)
- Fall  $kpos = 1$ : Er steht in Reihe acht oder eins, aber in keiner Ecke (b1 - g1 und b8 - g8)
- Fall  $kpos = 2$ : Er steht im Bauernterritorium
- Fall  $kpos = 3$ : Er ist fixiert. (Er steht auf e8)

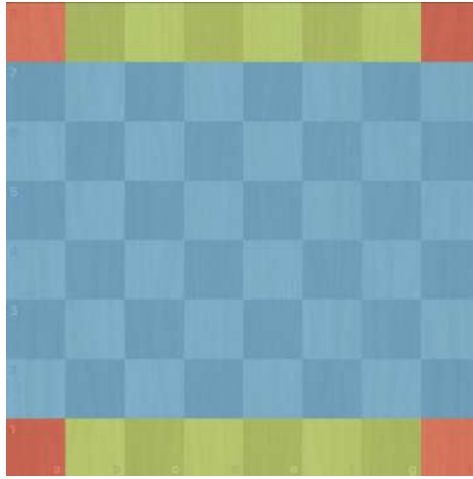


Abb. 25: Mögliche Positionen eines nicht-fixierten Königs. Markierungen:  
 $kpos = 0$ : rot,  $kpos = 1$ : grün,  $kpos = 2$ : blau.



Abb. 26: Mögliche Positionen eines fixierten Königs.  $kpos = 3$ : orange  
 markiert.

Es ist wichtig zu wissen, wie viele Möglichkeiten  $kmult$  es gibt, den schwarzen König innerhalb der einzelnen  $kpos$ -Fälle zu platzieren.

**Lemma 5.1** *Es gilt:*

$$kmult = \begin{cases} 4 - fwr & \text{für } kpos = 0 \\ 12 - 3 \cdot fixwk - fwr & \text{für } kpos = 1 \\ k & \text{für } kpos = 2 \\ 1 & \text{für } kpos = 3 \end{cases}$$

mit  $k = 48 - 4 \cdot fixp - wp - bp - 3 \cdot fixwk - fwr$

**Beweis** Wir werden nacheinander die einzelnen Fälle beweisen.

Fall  $kpos = 0$ :

Es gibt vier Ecken. Lediglich fixierte weiße Türme können Ecken vor der Platzierung des schwarzen Königs bereits besetzen. Deshalb gilt  $kmult = 4 - fwr$ .

Fall  $kpos = 1$ :

Es gibt in Reihe eins und acht jeweils sechs Felder, die keine Ecken sind. Zusammen ergibt das zwölf Felder. Lediglich ein fixierter weißer König kann eines dieser Felder - das Feld e1 - zu diesem Zeitpunkt bereits besetzen. Da Könige nicht nebeneinander stehen dürfen, kann der schwarze König in diesem Fall auch nicht die Felder d1 und f1 besetzen. Darüber hinaus darf der schwarze König nicht auf Feldern stehen, die direkt neben den fixierten weißen Türmen liegen (b1 und g1), da er sonst im Schach stehen würde. Dies bedeutet, dass das Nachbarfeld jedes fixierten weißen Turmes auch nicht besetzbar ist, demnach gilt  $kmult = 12 - 3 \cdot fixwk - fwr$ .

Fall  $kpos = 2$ :

In den Reihen zwei bis sieben befinden sich insgesamt 48 Felder. Von diesen wurden  $wp + bp$  Felder bereits von den weißen und schwarzen nicht-fixierten Bauern besetzt. Sollte es fixierte Bauern geben ( $fixp = 1$ ), können vier weitere Felder nicht vom schwarzen König besetzt werden. Diese sind die beiden Felder der fixierten Bauern, sowie die beiden Felder, über die der schwarze fixierte Bauer im letzten Zug hinweggezogen ist. Gibt es einen fixierten weißen König ( $fixwk = 1$ ), können die Felder d2, e2 und f2 nicht vom schwarzen König besetzt werden, da die beiden Könige nicht auf benachbarten Feldern stehen dürfen. Der schwarze König darf auch nicht auf den Nachbarfeldern von fixierten weißen Türmen stehen (diese sind a2 und h2). Demnach ist für jeden fixierten weißen Turm ein Feld weniger verfügbar. Damit bleiben für die Platzierung des schwarzen Königs  $kmult = 48 - 4 \cdot fixp - wp - bp - 3 \cdot fixwk - fwr$  Felder.

Fall  $kpos = 3$ :

Der König ist nach Definition auf das Feld e8 fixiert, somit gilt  $kmult = 1$ .  $\square$





Abb. 27: Es ist möglich, dass diese Felder, in den  $kpos$ -Fällen für den schwarzen König aufgrund von fixierten Figuren nicht verfügbar sind. Die fixierten Bauern können auch in anderen Spalten stehen. Markierungen:  
 $kpos = 0$ : rot,  $kpos = 1$ : orange,  $kpos = 2$ : blau

Nun wollen wir die Schach stellenden Figuren positionieren. Da es sehr aufwendig wäre, herauszufinden auf welchen Feldern die Schach stellende Figur stehen kann und wie viele Möglichkeiten es gibt, das Schach zu blockieren, betrachten wir nur Situationen, in denen es sicher ist, dass die Schach stellende Figur direkt neben dem schwarzen König stehen kann und somit auch kein Blockieren des Schachs möglich ist. Darüber hinaus soll die Schach stellende Figur nicht im Bauernterritorium stehen, da nicht für alle Bauernverteilungen gewährleistet werden kann, dass das Feld der Schach stellenden Figur nicht bereits durch einen Bauern besetzt ist. Gleiches gilt auch für den schwarzen König, weshalb wir keine Schachpositionen betrachten, bei denen der König im Bauernterritorium im Schach steht. Da Springer und Läufer im Bauernterritorium stehen müssen, um einen nicht im Bauernterritorium stehenden König Schach zu stellen, werden wir uns lediglich auf Damen und Türme fokussieren. Die Variable  $qspaces$  sagt uns, wie viele Möglichkeiten es gibt, den König im jeweiligen  $kpos$ -Fall durch Damen und Türme außerhalb des Bauernterritoriums Schach zu stellen.

**Lemma 5.2** Sei  $qspaces$  die Anzahl an Möglichkeiten den König mit einer Dame oder einem Turm auf einem benachbarten Feld Schach zu stellen, sodass weder die Schach stellende Figur noch der König im Bauernterritorium stehen. Es gilt:

$$qspaces = \begin{cases} 1 & \text{für } kpos = 0 \\ 2 & \text{für } kpos \in \{1, 3\} \\ 0 & \text{für } kpos = 2 \end{cases}$$

**Beweis** Durch die Bedingungen, dass der Turm bzw. die Dame auf benachbarten Feldern stehen müssen, die sich nicht im Bauernterritorium befinden, müssen der Turm bzw. die Dame in derselben Reihe wie der König stehen. Es reicht also zur Ermittlung von  $qspaces$  die Nachbarmfelder des Königs zu zählen, die sich in der jeweiligen Reihe befinden. Falls die Reihe im Bauernterritorium liegt ( $kpos = 2$ ), ist die Anzahl nach Definition null. Für  $kpos = 0$  gibt es ein Nachbarmfeld in derselben Reihe und für  $kpos \in \{1, 3\}$  gibt es zwei.  $\square$

Die verschiedenen Fälle werden in den folgenden Abbildungen veranschaulicht.



Abb. 28: Steht der König in einer Ecke ( $kpos = 0$ ), gibt es nur ein mögliches Feld für den Turm bzw. die Dame.



Abb. 29: Steht der König in der ersten oder letzten Reihe, aber nicht in einer Ecke ( $kpos \in \{1, 3\}$ ), gibt es zwei mögliche Felder für den Turm bzw. die Dame.



Abb. 30: Steht der König im Bauernterritorium ( $kpos = 2$ ), ist nicht sicher, dass Felder für den Turm bzw. die Dame frei sind, da die dargestellte Situation auftreten könnte. Deshalb rechnen wir konservativ mit  $qspaces = 0$ .

Da die Schachpositionen, in denen der schwarze König im Schach steht, illegal sind, dürfen keine weißen Damen bzw. weißen Türme auf den *qspaces* Nachbarfeldern des schwarzen Königs stehen.

**Satz 5.3** Sei

- $qspaces$  definiert wie in Lemma 5.2
- $kmult$  definiert wie in Lemma 5.1
- $space := 64 - 4 \cdot fixp - fixwk - fwr - fbr - wp - bp$

Dann ist  $K$  eine obere Schranke an die Anzahl an Möglichkeiten den schwarzen König auf einem Feld des vorliegenden kpos-Falls, sowie  $wD$  weiße Damen und  $wr$  weiße nicht-fixierte weiße Türme auf dem Schachbrett zu platzieren, mit

$$K = kmult \cdot \binom{space - 1 - qspaces}{wD, wr}$$

**Beweis**  $kmult$  beschreibt die Anzahl an Möglichkeiten den schwarzen König im vorliegenden kpos Fall zu platzieren. Die Variable  $space$  beschreibt die Anzahl an verfügbaren Feldern, nachdem die Bauern und alle fixierten Figuren bis auf einen fixierten schwarzen König platziert wurden. Der Multinomialkoeffizient beschreibt die Anzahl an Möglichkeiten die  $wD$  weißen Damen und  $wr$  nicht-fixierten Türme zu platzieren.  $\square$

Der Multiplikator  $multFWK$  ist eine obere Schranke an die Anzahl der Positionierungsmöglichkeiten des weißen Königs.

**Lemma 5.4** Sei  $space$  wie in Satz 5.3 definiert. Dann ist  $multFWK$  eine obere Schranke an die Anzahl der Positionierungsmöglichkeiten des weißen Königs, mit

$$multFWK = \begin{cases} 1 & \text{für } fwr > 0 \\ space - 1 - wr - wD & \text{für } fwr = 0 \end{cases}$$

**Beweis** Ist  $fwr > 0$ , ist der weiße König fixiert und somit gibt es nur eine Möglichkeit für die Positionierung des weißen Königs. Für  $fwr = 0$  wird der weiße König nach den Bauern, dem schwarzen König, den weißen Damen, den weißen Türmen und allen fixierten Figuren auf dem Schachbrett positioniert. Da  $space$  die Anzahl der verfügbaren Felder vor der Positionierung des schwarzen Königs, der  $wD$  weißen Damen und der  $wr$  nicht-fixierten weißen Türme ist, sind nach diesen Positionierungen noch maximal  $space - 1 - wr - wD$  Felder für die Positionierung eines nicht-fixierten weißen Königs verfügbar.  $\square$

Betrachten wir nun die Unterschiede zwischen meinem Code und der C-Übersetzung von Tromps Code. Alle Zeilenangaben beziehen sich auf meinen Code.

Schauen wir uns zunächst die Funktionen an, die in meinem Code hinzugekommen sind. Die Funktion `multinom` (Z. 163 bis 175) berechnet Multinomialkoeffizienten der Form  $\binom{n}{i,j}$  und speichert sie im Array `mn`. Die Funktion `binomial` (Z. 149-160) ist lediglich eine Hilfsfunktion, die Binomialkoeffizienten berechnet und im Array `binom` speichert. Die Funktion `multiplebinomrook` (Z. 178-196) berechnet die obere Schranke  $K$  aus Satz 5.3 und speichert sie im Array `mb`.

Fahren wir nun mit den Funktionen fort, die Änderungen erhalten haben. In der Funktion `count` wird in Zeile 222 gefragt, ob  $fbr = 0$  ist und es damit nach Definition keinen fixierten schwarzen König gibt. Falls dies der Fall ist, werden die Fälle  $kpos = 0$ ,  $kpos = 1$  und  $kpos = 2$  durchgegangen, andernfalls gilt  $kpos = 3$ . Wenn kein fixierter schwarzer König vorliegt, wird  $kmult$  in den Zeilen 262 bis 266, wie in Lemma 5.1 beschrieben, berechnet. Bei einem fixierten schwarzen König, wird  $kmult$  in Zeile 289 der Wert 1 zugewiesen. Die Variable `space` in den Zeilen 259 und 326 unterscheidet sich von der `space`-Variable aus Tromps Code dadurch, dass bei ihr `fixbk` noch nicht abgezogen wurde, da der schwarze König bei uns erst in der Funktion `multiplebinomrook` platziert wird (auch wenn er fixiert ist).

Während bei Tromp die Anzahl der möglichen Verteilungen aller Figuren nach der Platzierung der Bauern durch einen Multinomialkoeffizienten beschrieben wird, verwenden wir den Wert aus dem `mb`-Array für die Verteilung des schwarzen Königs, der weißen Damen und der weißen nicht-fixierten Türme, den Multiplikator `multFWK` für die Platzierung des weißen Königs und schließlich verteilen wir die restlichen Figuren mithilfe von Multinomialkoeffizienten, die mit dem `mn` Array realisiert werden.

$8.374.836.394.172.425.477.842.765.437.670.291.687.010.729.984 \approx 8,73 \cdot 10^{45}$  ist das Endergebnis unserer Berechnungen. Damit ist unsere obere Schranke um ungefähr 4,0 % kleiner als die von Tromp. Die einzelnen Teilergebnisse für die verschiedenen Werte von  $fwr$ ,  $fbr$  und  $fixp$  sind in der folgenden Abbildung zu sehen. Es ist zu beachten, dass diese Werte noch nicht mit zwei multipliziert wurden und somit nur die Anzahl angeben, wenn Weiß am Zug ist.

```
[Running] cd "/Users/christianbuchs/Documents/Studium/Bachelorarbeit/C Project/Folder/" && gcc arbeitsberei
Bachelorarbeit/C Project/Folder/"arbeitsbereich
fwr = 0 fbr = 0 fixp = 0 gerundet: 4.144773e+45 exakt: 4144773441728498560365341708306211071907069952
fwr = 0 fbr = 0 fixp = 1 gerundet: 3.066606e+43 exakt: 30666062795919153678752107982269681865064448
fwr = 0 fbr = 1 fixp = 0 gerundet: 5.737524e+42 exakt: 5737523849016796789065681742333186450391040
fwr = 0 fbr = 1 fixp = 1 gerundet: 4.532929e+40 exakt: 45329287090512824733318495978669733838848
fwr = 0 fbr = 2 fixp = 0 gerundet: 1.131520e+41 exakt: 113151969932855792273952879950937534234624
fwr = 0 fbr = 2 fixp = 1 gerundet: 8.620150e+38 exakt: 862015042677659888435339220996069523456
fwr = 1 fbr = 0 fixp = 0 gerundet: 5.912294e+42 exakt: 5912293887285700630466356389189630356881408
fwr = 1 fbr = 0 fixp = 1 gerundet: 4.588557e+40 exakt: 45885574713994993037604035509438063312896
fwr = 1 fbr = 1 fixp = 0 gerundet: 9.925824e+39 exakt: 9925823765983077419511628091318580805632
fwr = 1 fbr = 1 fixp = 1 gerundet: 8.352274e+37 exakt: 83522735541271808441166513994507223040
fwr = 1 fbr = 2 fixp = 0 gerundet: 1.994760e+38 exakt: 199476036685788573268802054854874759168
fwr = 1 fbr = 2 fixp = 1 gerundet: 1.620145e+36 exakt: 1620144867787475934843231477021278208
fwr = 2 fbr = 0 fixp = 0 gerundet: 1.123917e+41 exakt: 112391745590857961575565702234681938804736
fwr = 2 fbr = 0 fixp = 1 gerundet: 8.350204e+38 exakt: 835020432733113420235106718494434000096
fwr = 2 fbr = 1 fixp = 0 gerundet: 2.030196e+38 exakt: 203019648536401711584970237088844742656
fwr = 2 fbr = 1 fixp = 1 gerundet: 1.645616e+36 exakt: 1645615687922815973514991398727712768
fwr = 2 fbr = 2 fixp = 0 gerundet: 4.072906e+36 exakt: 4072906148386374497746531610594902016
fwr = 2 fbr = 2 fixp = 1 gerundet: 3.183536e+34 exakt: 31835362284437777765927429890637824
Totale Summe: 8374836394172425477842765437670291687010729984
Verbesserung um 4.0 Prozent
[Done] exited with code=1 in 21.318 seconds
```

Abb. 31: Ausgabe meines Programms

Die Laufzeit des Programms ist durch meine Erweiterung mehr als verdoppelt worden (von 9,2 auf 21,3 Sekunden). Obwohl es möglich ist, die obere Schranke noch zu verbessern, wird dies vermutlich nicht ohne eine starke Erhöhung der Laufzeit möglich sein, da immer mehr Fallunterscheidungen nötig sein werden, um die immer spezifischer einzugrenzenden illegalen Schachpositionen herauszufiltern.

Man kann das Programm weiter verbessern, indem man die pawns-Funktion so erweitert, dass sie für bestimmte Felder die Anzahl an Bauernverteilungen ausrechnet, in denen kein Bauer auf diesem Feld steht. Durch diese Anzahl kann zwischen den Fällen unterschieden werden, in denen das Feld frei ist und den Fällen, in denen das Feld blockiert ist. Dies kann dann verwendet werden, um den schwarzen König auch durch andere Figuren wie Springer oder Läufer Schach zu stellen und somit weitere illegale Schachpositionen herauszufiltern. Sollte es laufzeittechnisch möglich sein, dass sogar die Anzahl an Bauernverteilungen berechnet wird, in denen Kombination von Feldern unbesetzt sind, kann dies verwendet werden, um Schachpositionen herauszufiltern, in denen der schwarze König im Bauernterritorium Schach gestellt wird oder Schachpositionen bei denen zwischen der Schach stellenden Figur und dem schwarzen König mindestens ein Feld liegt.

Ich persönlich gehe nicht davon aus, dass die obere Schranke in den nächsten Jahren um mehr als 30% verbessert wird, da der Aufwand zur Implementierung weiterer Filter immer größer und der Ertrag immer kleiner wird. Verbesserungen in der Größenordnung von 10% bis maximal 20% halte ich für durchaus möglich.

## 6 Abschätzung durch ein Konfidenzintervall

Der Großteil von Tromps Arbeit besteht aus der Berechnung eines Konfidenzintervalls  $\Omega$ , in dem sich die Anzahl der legalen Schachpositionen  $L$  zum Konfidenzniveau von 95% befindet. Tromp führt diese Berechnung für mehrere Stichprobengrößen  $n$  durch. Unter anderem zieht er zwei Stichproben der Größe  $n = 1.000.000$  und fügt diese zu einer Stichprobe der Größe  $n = 2.000.000$  zusammen. Wir werden in diesem Kapitel zunächst die Realisierung der Stichprobe erklären, dann die mathematischen Hintergründe erläutern und zuletzt seine Ergebnisse für eine kleine Stichprobengröße  $n = 100$ , für eine mittlere Stichprobengröße  $n = 10.000$ , sowie für die Stichproben mit den Größen  $n = 1.000.000$  und  $n = 2.000.000$  betrachten. Alle Konzepte sind die Leistung von Tromp, sie wurden von mir jedoch deutlich verständlicher aufbereitet und erklärt, da seine Dokumentation oft nur sehr oberflächlich beschreibt und dem Leser in manchen Situationen viel Mitdenken überlässt.

### 6.1 Realisierung der Stichprobe

In diesem Unterkapitel sind die Definition des Rankings, die Definition der Multiplizitäten, die Erklärung der modifizierten FEN-Notation, die Beschreibung zur Generierung der Pseudozufallszahlen und die Beschreibung der Generierung der legalen Urpositionen für den Test der Invertierbarkeit von Tromp. Der Rest ist meine Eigenleistung.

Tromp realisiert eine Stichprobe der Größe  $n$ , indem er mithilfe von Haskell  $n$  Pseudozufallszahlen generiert und diese in Urpositionen überführt. Wie werden später darauf eingehen, was Urpositionen sind. Die Umwandlung von Pseudozufallszahlen in Urpositionen und umgekehrt von Urpositionen zu Pseudozufallszahlen geschieht mithilfe eines invertierbaren Rankings.

**Definition 6.1 (Ranking)** *Ein Ranking  $R$  für eine Menge  $G$  ist ein Tupel  $(size, u, r, c)$  mit*

- *der Größe der Menge  $G$ :  $|G| = size \in \mathbb{N}$*
- *einer unrank-Funktion  $u : \{0, \dots, size - 1\} \rightarrow G$*
- *einer rank-Funktion  $r : G \rightarrow \{0, \dots, size - 1\}$*
- *einer Vergleichsfunktion  $c$ , die je zwei Zahlen (gerankte Objekte) vergleicht.*

*Ein Ranking ist invertierbar, wenn  $r$  injektiv ist.*

**Bemerkung** Zu Beginn seiner Dokumentation definiert Tromp ein Ranking zunächst nur als 3-Tupel bestehend aus *size*, einer rank-Funktion und einer unrank-Funktion. Diese Definition erweitert er später um die Vergleichsfunktion *c*, damit er die Rankings sortieren und somit besser auf Invertierbarkeit überprüfen kann.

Die Invertierbarkeit ist wichtig, damit nicht zwei verschiedene Urpositionen der selben Zahl zugewiesen werden.

Es ist nicht sinnvoll die Menge der Schachpositionen als Grundgesamtheit für die Stichprobe zu nehmen, da die Menge der Schachpositionen nach [3]  $13^{64} \cdot 2 \cdot 9 \cdot 2^4 = 5,6 \cdot 10^{73}$  Elemente hat. Das heißt es gibt mindestens  $6,7 \cdot 10^{27}$  mal so viele illegale Schachpositionen wie legale Schachpositionen. Deshalb wäre eine immense Stichprobengröße nötig, um relevante Aussagen treffen zu können. Moderne Computer verfügen jedoch nicht über genügend Rechenleistung, um derart viele Schachpositionen auf Legalität prüfen zu können.

Sei  $U$  die Menge aller Positionen, die bei der Berechnung der oberen Schranke in Kapitel vier mitgezählt worden sind. Wir benutzen hier bewusst nicht das Wort Schachpositionen, da es bei der Berechnung der oberen Schranke Schachpositionen gab, die mehrfach mitgezählt wurden, wie wir gleich sehen werden. Die Menge  $U$  bietet sich deutlich eher als Grundgesamtheit an, da sie nur  $8,7 \cdot 10^{45}$  Elemente besitzt. Deshalb hat Tromp die Urpositionen (Abkürzung für uniquely rankable positions) eingeführt, deren Grundgesamtheit eben die Menge  $U$  ist. Urpositionen sind Schachpositionen mit einigen Zusatzinformationen, die dazu führen, dass jede Urposition in der oberen Schranke von Kapitel vier (also der Mächtigkeit von  $U$ ) genau einmal mitgezählt wurde. Demnach ist  $U$  die Menge aller Urpositionen und die Variable *size* des Rankings ist die in Kapitel vier berechnete obere Schranke.

Weiter oben haben wir erwähnt, dass einige Schachpositionen bei der Berechnung der oberen Schranke von Kapitel vier mehrfach mitgezählt wurden, dies wollen wir nun zeigen. Oder anders ausgedrückt: Wir wollen zeigen, dass es Schachpositionen gibt, die zu mehreren Urpositionen gehören. Die Schachposition, die wir uns in diesem Beispiel anschauen möchten, ist durch  $fixbr = 1$ ,  $fixwr = 1$ , "der Bauer auf d5 kann En-Passant geschlagen werden", "Weiß ist am Zug" und die Figurenverteilung in der folgenden Abbildung eindeutig beschrieben. Eine der Zusatzinformationen einer Urposition sind die Felder der fixierten Bauern. Es gibt mehrere Urpositionen, die dieser Schachposition entsprechen, da sowohl der weiße Bauer auf c5 als auch der weiße Bauer auf e5 der weiße fixierte Bauer sein kann. Auch im Programmcode werden beide Urpositionen durch den Multiplikator *multEP* einzeln mitgezählt, obwohl beide Urpositionen dieselbe Schachposition beschreiben. Die Anzahl an Urpositionen, die dieselbe Schachposition  $p$  beschreiben, wird Multiplizität von  $p$  genannt und wir schreiben  $m(p)$ . Eine Urposition hat die Multiplizität  $m$ , wenn die zugehörige Schachposition die Multiplizität  $m$  hat.





Abb. 32: Diese Figurenverteilung gehört zu einer Schachposition, die von mehr als einer Urposition beschrieben wird.

Da ein zentraler Teil der Auswertung der Stichprobe die Prüfung der Urpositionen auf Legalität ist, müssen die Urpositionen effizient ein- und ausgegeben werden. Dafür verwendet Tromp die FEN-Notation mit folgenden Modifikationen:

- Das En-Passant-Feld der FEN-Notation wird nur genutzt, wenn auch ein zu schlagender Bauer verfügbar ist (**blau**).
- Es werden zwei Felder am Ende hinzugefügt. Diese beschreiben die Multiplizität der Schachposition (**türkis**) und die Nummer der Urposition in einer Nummerierung aller Urpositionen, die zu dieser Schachposition gehören (**orange**).

Das ist ein Beispiel für die modifizierte FEN-Notation:

r3kb1r/ppp1pp1p/3p4/5PpP/8/8/PPPPP1P1/RNBQKBNR w KQkq **!g6** **99**  
140

Wie gesagt, werden die modifizierten FEN-Notationen zur Eingabe der Urpositionen verwendet. Unter anderem auch zur Eingabe in die Legalitätsprogramme. Dies zeigt, dass für die Legalität einer Urposition die Zusatzinformationen irrelevant sind, da sie gar nicht in der FEN-Notation beschrieben werden. Lediglich die Information über die Multiplizität und die Nummer in einer Rangfolge kommen hinzu. Daraus folgt, dass eine Urposition genau dann legal ist, wenn die zugehörige Schachposition legal ist.

Tromp testet, ob das von ihm verwendete Ranking invertierbar ist, indem er legale Urpositionen generiert und diese dann rankt und unrankt. Anschließend

sortiert er sie und überprüft am Ende, ob die Urpositionen vor dem ranken und unranken mit denen danach übereinstimmen. Für die Generierung der legalen Urpositionen ließ er ein Programm Schach spielen, indem es jeweils einen aus allen möglichen Zügen auswählt und nach jedem Zug mit einer Wahrscheinlichkeit von 1 % abbricht. Die Sortierung der Ergebnisse erfolgte mit der Vergleichsfunktion. Dieser Test ergab, dass sein Ranking invertierbar ist. Darüber hinaus verwendet er auch einige Stichproben, um die Invertierbarkeit mehrfach zu überprüfen.

Für die Stichproben verwendet Tromp Pseudozufallszahlen, die mithilfe des Zufallszahlengenerators der Haskell-Programmiersoftware generiert wurden. Die Prüfung der Legalität der Positionen gelang mithilfe verschiedener Programme, die Illegalität bzw. Legalität beweisen können. Vor allem bei den größeren Stichproben blieben einige Urpositionen übrig, die von den Programmen weder als legal noch als illegal eingeordnet werden konnten. Diese mussten manuell auf Legalität geprüft werden.

## 6.2 Mathematischer Hintergrund

In diesem Abschnitt werden wir Tromps Beweis dafür betrachten, dass er einen erwartungstreuen Schätzer verwendet. Darüber hinaus zeigt Tromp undetailliert, dass  $Y(S)$  approximativ normalverteilt ist. Diesen Beweis werden wir etwas ergänzen. Für diesen Abschnitt führen wir folgende Variablen ein: Sei

- $I_m$  die Anzahl an legalen Schachpositionen mit Multiplizität  $m$
- $n_m$  die Anzahl an Urpositionen mit Multiplizität  $m$
- $L = \sum_{m \in \mathbb{N}} I_m$  die Anzahl an legalen Schachpositionen
- $LU$  die Menge der legalen Urpositionen
- $N = \sum_{m \in \mathbb{N}} n_m$  die Anzahl an Urpositionen
- $X \in U$ , dann ist  $m(X)$  die Multiplizität von  $X$ .
- $X \in U$  und sei  $m$  die Multiplizität von  $X$ . Dann ist  $Y(X)$  eine Zufallsvariable und es gilt:

$$Y(X) = \begin{cases} 0 & \text{für } X \text{ illegal} \\ \frac{N}{m} & \text{für } X \text{ legal} \end{cases}$$

**Satz 6.2** *Es gilt:  $Y$  ist ein erwartungstreuer Schätzer von  $L$*

**Beweis** Wir beweisen den Satz durch die folgende Gleichung:

$$\mathbb{E}[Y] = \sum_{X \in U} P(X \in LU \wedge M(X) = m) \cdot Y(X) = \sum_{m \in \mathbb{N}} \frac{m \cdot I_m}{N} \cdot \frac{N}{m} = \sum_{m \in \mathbb{N}} I_m = L$$

Das erste Gleichheitszeichen folgt aus der Definition des Erwartungswerts. Das zweite Gleichheitszeichen kann mit der folgenden Begründung nachvollzogen werden. Es gibt  $I_m \cdot m$  Urpositionen, die sowohl Multiplizität  $m$  haben, als auch legal sind und es wird aus  $N$  möglichen Urpositionen "gezogen", sodass die Wahrscheinlichkeit  $\frac{m \cdot I_m}{N}$  ist eine solche Urposition zu "ziehen".  $\square$

Für eine Stichprobe  $S$  der Größe  $n$  mit  $n_s$  legalen Urpositionen schätzt Tromp  $L$  als den Durchschnitt der einzelnen Schätzungen und somit gilt:

$$Y(S) = \frac{1}{n} \cdot \sum_{u \in S \cap LU} \frac{N}{m(u)} = \frac{n_s}{n} \cdot N \cdot \frac{1}{n_s} \cdot \sum_{u \in S \cap LU} \frac{1}{m(u)} \quad (3)$$

Für jede legale Schachposition ist auch jede zugehörige Urposition legal. Da eine legale Schachposition mit mehr zugehörigen Urpositionen deshalb eher in einer Stichprobe auftaucht, ist deshalb nicht nur die reine Anzahl an legalen Urpositionen wichtig, sondern auch die Multiplizität eben jener. Dies kann man in Formel (3) anhand des Terms

$$\frac{1}{n_s} \cdot \sum_{u \in S \cap LU} \frac{1}{m(u)}$$

sehen. Dieser Term gibt die inverse durchschnittliche Multiplizität der legalen Urpositionen ( $IDM$ ) an. Je kleiner die Multiplizität der Urpositionen ist, desto größer ist die  $IDM$  und damit der Wert  $Y(S)$ , da es pro legaler Schachposition weniger Urpositionen gibt und es somit bei gleicher Anzahl an legaler Urpositionen mehr legale Schachpositionen gibt. Im nachfolgenden Satz sehen wir, dass  $Y(S)$  approximativ normalverteilt ist.

**Satz 6.3** Sei  $Y(S)$  wie in (3) definiert. Dann ist  $Y(S)$  approximativ normalverteilt.

**Beweis** Wir wissen bereits, dass  $\mathbb{E}[Y] = L < \infty$ . Da sowohl  $1 \leq Y \leq N$  als auch  $1 \leq L \leq N$  gilt, ist auch  $Var(Y) = \mathbb{E}((Y - L)^2) < \infty$ . Sei die Stichprobe  $S$  bestehend aus  $n$  Urpositionen  $(X_1, \dots, X_n)$ . Es ist offensichtlich, dass die  $Y(X_i)$  mit  $i \in \{1, \dots, n\}$  unabhängig und identisch verteilt sind und da sowohl der Erwartungswert als auch die Varianz endlich sind, folgt aus dem Zentralen Grenzwertsatz, dass  $Y(S)$  approximativ normalverteilt ist.  $\square$

Diese Approximative Normalverteilttheit hilft Tromp bei der Berechnung der Konfidenzintervalle. Die Berechnungen erfolgen in der Datei *estimate.pl*.

### 6.3 Ergebnisse der Stichproben

Alle Ergebnisse stammen, sofern nichts anderes erwähnt wird, von Tromp. Die Analysen sind meine Eigenleistung.

In der kleinen Stichprobe  $S1$  mit der Größe  $n = 100$ , gibt es  $n_{S1} = 3$  legale Urpositionen, die alle eine Multiplizität von 1 besitzen und somit  $IDM = 1$  gilt. Daraus errechnet Tromp, dass das Konfidenzintervall zum Konfidenzniveau von 95% die Intervallgrenzen  $(2,6 \pm 2,9) \cdot 10^{44}$  besitzt. Bereits hier wird klar, dass alle oberen Schranken um ein Vielfaches zu groß sind.

In der mittleren Stichprobe  $S2$  mit der Größe  $n = 10.000$  waren  $n_{S2} = 538$  Urpositionen legal und die  $IDM \approx 0,97847$ . Das Konfidenzintervall zum Konfidenzniveau von 95% hat die Intervallgrenzen  $(4,59 \pm 0,38) \cdot 10^{44}$ . Auffällig ist, dass die relative Häufigkeit an legalen Urpositionen in dieser Stichprobe mit 5,38 % beinahe doppelt so hoch ist wie in der Stichprobe  $S1$ , die bei 3,00 % lag. Dies lässt vermuten, dass die kleinere Stichprobe unterdurchschnittlich viele legale Urpositionen enthalten hat. Die Unterrepräsentation der legalen Urpositionen in der kleineren Stichprobe wird etwas durch die größere  $IDM$  ausgeglichen.

Tromp zog zwei Stichproben der Größe  $n = 1.000.000$ . Die erste Stichprobe  $S3$  wurde mit der Version 1.1 des Random-Pakets von Haskell und die zweite Stichprobe  $S4$  mit der Version 1.2 des selben Pakets erstellt. Abschließend hat er beide Stichproben zu einer Stichprobe  $S5$  der Größe  $n = 2.000.000$  zusammengeführt. Wir werden uns die Auswertung von  $S3$  etwas genauer anschauen und bei  $S4$  und  $S5$  lediglich die Ergebnisse betrachten.

Die erste Analyse von  $S3$  ergab:

```
1 $ src/legal < sortedRndImFENS | cut -f2 -d $'\t' | sort | uniq -c |
   sort -rn
2 492045 Illegal Both Kings in Check
3 173401 Illegal Side not to move in Check
4 102541 Illegal Adjacent Kings
5 79019 Illegal Bishops Too Monochromatic
6 53063 Single Check
7 35584 No Checks
8 29962 Illegal Triple Check
9 27415 Illegal Double Check
10 6256 Discovered Double Check
11 714 Illegal Double Knight Check
```

Listing 1: Ergebnisse der Erstanalyse von  $S3$

Lediglich die Situationen, in denen ein Single Check, No Check oder Discovered Double Check vorliegt können legal sein. Diese 94.903 Urpositionen wurden in das Texel-Programm von Peter Österlund überführt. Nach der Auswertung durch das Texel-Programm blieben 65 Urpositionen übrig, die manuell überprüft werden mussten.

$S3$  enthält  $n_{S3} = 56.011$  legale Urpositionen und hat die  $IDM \approx 0,98013$ . Das Konfidenzintervall zum Konfidenzniveau von 95% hat die Intervallgrenzen  $(4,79082 \pm 0,0387698) \cdot 10^{44}$ . Die Werte von  $S2$  und  $S3$  weisen außer eines

zehnfach kleineren Abstands der Intervallgrenzen des Konfidenzintervalls von  $S3$  kaum immense Abweichung auf. Diese verbesserte Genauigkeit ist durch die 100-fache Stichprobengröße von  $S3$  aber auch nicht ungewöhnlich.

Zur Stichprobe  $S4$  hat Tromp leider deutlich weniger Daten zur Verfügung gestellt, jedoch konnte ich einige Daten von  $S4$  aus den Werten von  $S3$  und  $S5$  errechnen. In  $S4$  sind  $n_{S4} = 56.743$  Urpositionen legal und die  $IDM$  liegt bei ungefähr 0,98006. Das Konfidenzintervall zum Konfidenzniveau von 95% hat die Intervallgrenzen bei  $(4,85304 \pm 0,039004) \cdot 10^{44}$ . Die Werte der Stichproben  $S3$  und  $S4$  weichen nur marginal voneinander ab.

In der zusammengeführten Stichprobe  $S5$  sind  $n_{S5} = 112.754$  Urpositionen legal und die  $IDM$  liegt bei ungefähr 0,98010. Die Grenzen des Konfidenzintervalls zum Konfidenzniveau von 95% liegen bei  $(4,82193 \pm 0,0274973) \cdot 10^{44}$ . Durch das Zusammenführen der Stichproben  $S3$  und  $S4$  konnte der Abstand der Intervallgrenzen des Konfidenzintervalls noch einmal um ungefähr 30% verringert werden.

Dies ist die Ausgabe der estimate.pl-Datei:

```

1 $ cat Texel.out.94903 Texel.out.95544 | src/estimate.pl 2000000
2 Exp[Y] ~ 0.0552548 * N
3 Var[Y] ~ 0.051689 * N^2
4 Sigma[Y] ~ 0.227352 * N
5 Sigma[Y[S]] ~ 0.000160762 * N
6 n = 2000000 samples 112754/190447 legal 108547 3418 705 56 3 23
  2 avg 1/mult 0.980095015106633
7 estimate 4.82193e+44 +- 2.74973e+42 at 95% confidence

```

Listing 2: Ausgabe der estimate.pl-Datei für  $S5$

Die Anzahl an legalen Schachpositionen  $L$  ist ungefähr  $4,82 \cdot 10^{44}$  und  $L$  liegt mit einer Wahrscheinlichkeit von 95% im Intervall  $(4,82 \pm 0,03) \cdot 10^{44}$ . Die obere Schranke  $8,73 \cdot 10^{45}$  ist ziemlich genau 18 mal so groß, wie die obere Grenze des Konfidenzintervalls.

## 7 Fazit

Durch die obere Schranke  $8,73 \cdot 10^{45}$  und das Konfidenzintervall  $(4,82 \pm 0,03) \cdot 10^{44}$  können wir die Anzahl der legalen Schachpositionen bereits ziemlich genau voraussagen. Dies ist vor allem dann bemerkenswert, wenn man bedenkt, dass die Anzahl an Schachpositionen bei  $5,6 \cdot 10^{73}$  liegt [3].

Es ist zwar zu erwarten, dass in Zukunft sowohl die obere Schranke als auch die Genauigkeit des Konfidenzintervalls verbessert wird, jedoch ist noch offen, wie stark diese Verbesserungen ausfallen werden. Da es momentan sowohl für die Berechnung von oberen Schranken als auch für die Auswertung von Stichproben begrenzende Faktoren gibt.

Bei der Auswertung der Stichproben ist momentan vor allem die Abdeckung durch die Legalitätsprogramme der begrenzende Faktor. In der Stichprobe S3 mussten 65 der 1.000.000 Urpositionen manuell überprüft werden. Wenn wir davon ausgehen, dass die relative Häufigkeit von manuell zu überprüfenden Urpositionen für jede Stichprobe ungefähr gleich groß ist, müssten bei einer Stichprobe mit  $n = 10^7$  Elementen bereits 650 und bei einer Stichprobe mit  $n = 10^9$  Elementen bereits 65.000 Urpositionen manuell überprüft werden. Deshalb sind signifikante Verbesserungen der Genauigkeit des Konfidenzintervalls nur dann möglich, wenn die Legalitätsprogramme deutlich mehr Urpositionen abdecken.

Bei der Berechnung einer oberen Schranke sind sowohl Aufwand-Ertrags-Verhältnis als auch die Leistungsfähigkeit der Computer limitierende Faktoren. Will man eine signifikante Verbesserung erreichen, ist es notwendig noch viel genauer darauf einzugehen, welche Figuren auf welchen Feldern wie häufig stehen. Dies benötigt sehr viele Fallunterscheidungen und erhöht die Laufzeit des Programms erheblich. Darüber hinaus werden mit der Zeit immer mehr Fallunterscheidungen nötig, um eine kleiner werdende Anzahl an illegalen Schachpositionen herauszufiltern. Sofern kein neuer Ansatz gefunden wird, ist es meiner Meinung nach unwahrscheinlich, dass meine obere Schranke in den nächsten Jahren um mehr als 30% verbessert wird.

Wir wollen noch Shannons Schätzwert von  $10^{43}$  lobend erwähnen, der trotz der Simplität seiner Berechnung nur ungefähr um den Faktor 48 zu klein ist.

Abschließend lässt sich also feststellen, dass uns Computer ermöglichen, einen sehr guten Schätzwert für die Anzahl an legalen Schachpositionen zu finden, aber sehr wahrscheinlich niemals die genaue Anzahl berechnen können. Genauso wie heutzutage Computer Schach besser als die besten Großmeister spielen, aber niemals perfekt spielen werden.

Aber das ist ja auch der Reiz daran sowohl in der Mathematik als auch im Schach.

## 8 Quellenverzeichnis

- [1] Chinchalkar, Shirish: An upper bound for the number of reachable positions, ICGA Journal, Vol. 19, No. 3 S.181-183, 1996, 3 Seiten
- [2] Entriiken, William: Github Post chess-upper-bound-armies, Version vom 07.08.2016, <https://github.com/fulldecent/chess-upper-bound-armies>, Zugriffszeit: 09.09.2023 13.47 CEST
- [3] Österlund, Peter: Github Post texel, Version vom 25.09.2023, <https://github.com/peterosterlund2/texel/blob/master/doc/proofgame.md>, Zugriffszeit: 28.09.2023 13:57 CEST
- [4] Shannon, Claude E. : XXII. Programming a Computer for Playing Chess, Philosophical Magazine, Ser.7, Vol. 41, No. 314 , 1950, 18 Seiten
- [5] Tromp, John: Github Post ChessPositionRanking, Version vom 12.05.2022, <https://github.com/tromp/ChessPositionRanking>, Zugriffszeit: 09.09.2023 18:22 CEST

Bildquellen: Alle verwendeten Bilder sind von mir getätigte Screenshots. Die Schachpositionen wurden mit der Website <https://www.chess.com> erstellt.

## 9 Anhang

Im Anhang sind die Codes der folgenden Programme zu finden:

- Die C-Übersetzung des Haskell-Codes von Tromp *kopievtromp.c*
- Meine Erweiterung *meincode.c*



## kopievontromp.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  extern double pow(double, double);
6
7  #define min(a,b) ((a) < (b) ? (a) : (b))
8  #define max(a,b) ((a) > (b) ? (a) : (b))
9
10
11 double fac[65];          // fac[n] = n!
12 double choose2 [49][9][9]; // choose[n][j][k] = Multinomialkoeffizient  $n! / (j! * k! * (n - j - k)!)$ 
13 double mopps_[9][33][9]; // Umsetzung von m(p,s,n) aus Satz 4.10
14 double pawns[2][9][9][9];
15
16
17 //Berechnet die Werte von fac[i]
18 void factorial() {
19     int i, j;
20     for (i=0; i<65; i++) {
21         fac[i] = 1.0;
22         for (j=2; j<=i; j++) {
23             fac[i] *= j;
24         }
25     }
26 }
27
28 // Berechnet die Werte von choose2[n][k1][k2]
29 void choose2function(){
30     int n, k1, k2;
31     for (n = 0; n <= 48; n++){
32         for (k1 = 0; k1 <= 8; k1++){
33             for (k2 = 0; k2 <= 8; k2++){
34                 if (n >= k1 + k2){
35                     choose2[n][k1][k2] = fac[n] / (fac[k1] * fac[k2] * fac[n - k1 - k2]);
36                 } else {
37                     choose2[n][k1][k2] = 0;
38                 }
39             }
40         }
41     }
42 }
43
44 //Berechnet die Werte von mopps[p][s][n]
45 double opps(int p, int s, int n) {
46     double res = 0.0;
47     if (p == 0 && s == 0) {
48         res = 1;
49     } else if (p == 0 || n == 0) {
50         res = 0;
51     } else {
52         res = mopps_[p][s][n - 1];
53         for (int i = 0; i <= (s < 4 ? s : 4); ++i) {
54             res += (5 - i) * mopps_[p - 1][s - i][n - 1];
55         }
56     }
57     return res;
58 }
59
60 //Weißt mopps[p][s][n], die in opps(p,s,n) berechneten Werte zu
61 void moppsfunction(){
62     double max = 0.0;
63     for (int p = 0; p <= 8; ++p) {
64         for (int s = 0; s <= 32; ++s) {
65             for (int n = 0; n <= 8; ++n) {
66                 mopps_[p][s][n] = opps(p, s, n);
67             }
68         }
69     }
70 }
71

```

```

72 // Berechnet die innere Summe in Satz 4.12
73 // dw = 1 <=> weißer Bauer in schwarzer fixierter Spalte
74 // db = 1 <=> schwarzer Bauer in weißer fixierter Spalte
75 // ds = ds1 + ds2 = Anzahl an eingeschlossener Felder in fixierten Spalten
76 double pawnsep(int dw, int db, int ds, int wp, int bp, int opp){
77     int opp_ = opp - dw - db;           // Mindestanzahl blockierter nicht-fixierter Spalten
78     int s;                             // Anzahl eingeschlossener Felder
79     double res = 0.0;
80     for (s = 0; s <= 4 * opp_; s++){
81         if(bp + dw >= opp && wp + db >= opp){ // Damit keine negative (also nichtdefinierte) Komponente von choose2 aufgerufen wird
82             res += mopps_[opp_][s][6] * choose2[44 - opp - opp_ - ds - s][wp + db - opp][bp + dw - opp];
83         }
84     }
85     return res;
86 }
87
88
89 void pawnsfunction(){
90     int s, wp, bp, opp;
91     int ds1, ds2;
92     double res;
93
94     // Fall: Kein Enpassant -> fixp = 0
95     // Berechnet obere Schranke aus Satz 4.11
96     for (wp = 0; wp <= 8; wp++){
97         for (bp = 0; bp <= 8; bp++){
98             for (opp = 0; opp <= 8; opp++){
99                 if (opp > min(wp, bp)){ // Es darf nicht mehr blockierte Spalten als Bauern einer Seite geben
100                     pawns[0][wp][bp][opp] = 0;
101                 } else {
102                     res = 0.0;
103                     for (s = 0; s <= 4 * opp; s++){
104                         res += mopps_[opp][s][8] * choose2[48 - 2 * opp - s][wp - opp][bp - opp];
105                     }
106                     pawns[0][wp][bp][opp] = res;
107                 }
108             }
109         }
110     }
111
112     // Fall: Enpassant -> fixp = 1
113     // Berechnet obere Schranke aus Satz 4.12
114     for (wp = 0; wp <= 8; wp++){
115         for (bp = 0; bp <= 8; bp++){
116             for (opp = 0; opp <= 8; opp++){
117                 res = 0;
118                 if (opp > min(wp + 1, bp + 1)){ // Es darf nicht mehr blockierte Spalten als Bauern einer Seite geben
119                     pawns[1][wp][bp][opp] = 0;
120                 } else {
121                     res = pawnsep(0, 0, 0, wp, bp, opp);
122                     for (ds1 = 0; ds1 <= 2 && opp > 1; ds1++){
123                         for (ds2 = 0; ds2 <= 1; ds2++){
124                             res += pawnsep(1, 1, ds1 + ds2, wp, bp, opp);
125                         }
126                     }
127                     for (ds1 = 0; ds1 <= 2 && opp > 0; ds1++){
128                         res += pawnsep(1, 0, ds1, wp, bp, opp);
129                     }
130                     for (ds2 = 0; ds2 <= 1 && opp > 0; ds2++){
131                         res += pawnsep(0, 1, ds2, wp, bp, opp);
132                     }
133                     pawns[1][wp][bp][opp] = res;
134                 }
135             }
136         }
137     }
138 }
139
140 // Berechnet eine obere Schranke (bis auf Multiplikatoren) für die Anzahl an Schachpositionen, bei denen Weiß am Zug ist
141 // in Abhängigkeit von fwr, fbr und fixp
142 double count(int fwr, int fbr, int fixp){
143     double summe = 0.0;
144     double h = 0.0; // Hilfsvariable, damit der Code nicht zu weit nach rechts geht & gedruckt werden kann
145     int np = 8 - fixp; // Maximal mögliche nicht-fixierte Bauern pro Seite
146     int fixwk = (fwr != 0 ? 1 : 0); // Es existiert ein fixierter König <=> Mind. 1 Fixierter Turm existiert
147     int fixbk = (fbr != 0 ? 1 : 0);

```

```

148 int uwr, wk, wq, wnp1, wr, wb, wnp2, wnp3, wn, wnp4, wproms, wp, wpx;
149 int ubr, bk, bq, bnp1, br, bb, bnp2, bnp3, bn, bnp4, bproms, bp, bpx;
150 int wpcs, bpcs, caps, maxuwp, maxubp, minopp, space;
151
152 // wprod und bprod sind Produkte von Fakultäten und dienen zur Berechnung eines Multinomialkoeffizienten
153 double wprod, bprod;
154
155 // Anzahl der weißen Figuren
156 uwr = 2 - fwr; // Da es 2 Türme zum Start gibt, müssen mind. 2 - fwr Türme nicht-fixiert sein.
157 wk = floor(uwr / 2); // Anzahl nicht-fixierter Könige
158
159
160 // Die Verringerung der Variable np zu wnp1 zu Variablen wie wnp4 hilft Figurenkombinationen herauszufiltern,
161 // für die zu viele Promotionen nötig wären.
162
163 for (wq = 0; wq <= 1 + np; wq++){ // wq = Anzahl weißer Damen
164     wnp1 = np - max(wq - 1, 0);
165     for (wr = 0; wr <= uwr + wnp1; wr++){ // wr = Anzahl weißer nicht-fixierter Türme
166         wnp2 = wnp1 - max(wr - uwr, 0);
167         for (wb = 0; wb <= 2 + wnp2; wb++){ // wb = Anzahl weißer Läufer
168             wnp3 = wnp2 - max(wb - 2, 0);
169             for (wn = 0; wn <= 2 + wnp3; wn++){ // wn = Anzahl weißer Springer
170                 wnp4 = wnp3 - max(wn - 2, 0);
171
172                 wproms = np - wnp4; // wproms = Anzahl weißer Promotionen
173
174                 wprod = fac[wk] * fac[wq] * fac[wr] * fac[wb] * fac[wn]; // Dient zur Berechnung eines Multinomialkoeff.
175
176                 for (wp = 0; wp <= wnp4; wp++){ // wp = Anzahl nicht-fixierter weißer Bauern
177                     wpx = np - wp - wproms; // wpx = Anzahl geschlagener weißer Bauern
178
179                     wpcs = wk + wq + wr + wb + wn; // wpcs = Anzahl weißer Pieces mit nichtfix König
180
181                     // Anzahl der schwarzen Figuren
182                     ubr = 2 - fbr; // Mindestanzahl nicht-fixierter schwarzer Türme
183                     bk = floor(ubr / 2); // Anzahl nicht-fixierter schwarzer Könige
184
185                     for (bq = 0; bq <= 1 + np; bq++){ // bq = Anzahl schwarzer Damen
186                         bnp1 = np - max(bq - 1, 0);
187                         for (br = 0; br <= ubr + bnp1; br++){ // br = Anzahl schwarzer nicht-fixierter Türme
188                             bnp2 = bnp1 - max(br - ubr, 0);
189                             for (bb = 0; bb <= 2 + bnp2; bb++){ // bb = Anzahl schwarzer Läufer
190                                 bnp3 = bnp2 - max(bb - 2, 0);
191                                 for (bn = 0; bn <= 2 + bnp3; bn++){ // bn = Anzahl schwarzer Springer
192                                     bnp4 = bnp3 - max(bn - 2, 0);
193
194                                     bproms = np - bnp4; // bproms = Anzahl schwarzer Promotionen
195
196                                     bprod = fac[bk] * fac[bq] * fac[br] * fac[bb] * fac[bn];
197
198                                     for (bp = 0; bp <= bnp4; bp++){ // bp = Anzahl nicht-fixierter schwarzer Bauern
199                                         bpx = np - bp - bproms; // bpx = Anzahl geschlagener schwarzer Bauern
200
201                                         bpcs = bk + bq + br + bb + bn; // bpcs = Anzahl schwarzer Pieces & nicht-fix. König
202
203                                         // Ist der weiße König fixiert ist fixwk = 1,
204                                         // sonst ist der nicht-fixierte weiße König, in wpcs enthalten. Für Schwarz analog
205                                         // caps = 32 - alle noch vorhandenen Figuren
206                                         caps = 32 - 2 * fixp - fixwk - fixbk - fwr - fbr - wp - bp - wpcs - bpcs;
207
208                                         maxuwp = bpx + caps - wproms; // Maximalanzahl weißer Bauern in freien Spalten
209                                         maxubp = wpx + caps - bproms; // Maximalanzahl schwarzer Bauern in freien Spalten
210                                         h = 0.0;
211                                         // Testet die Bedingung aus Lemma 4.7
212                                         if (maxuwp < 0 || maxubp < 0){
213                                             continue;
214                                         } else {
215                                             // Mindestanzahl blockierter Spalten
216                                             minopp = max(0, fixp + wp - maxuwp);
217                                             space = 64 - 4 * fixp - fixwk - fixbk - fwr - fbr - wp - bp;
218                                             h += pawns[fixp][wp][bp][minopp];
219                                             h *= fac[space] / (fac[space - wpcs - bpcs] * wprod * bprod);
220                                             summe += h;
221                                         }
222                                     }
223                                 }
224                             }
225                         }
226                     }
227                 }
228             }
229         }
230     }
231 }

```

```

224     }
225     }
226     }
227     }
228     }
229     }
230     }
231     }
232     }
233 }
234     return summe;
235 }
236
237 int main (){
238
239     //Initialisierung der Arrays
240     factorial();
241     choose2function();
242     moppsfunction();
243     pawnsfunction();
244     int fwr, fbr, fixp;
245     double res = 0.0;
246     double summe = 0.0;
247     double multFWR, multEP, multFBR;
248
249     // Alle möglichen Werte für fwr, fbr und fixp durchlaufen
250     for (fwr = 0; fwr <= 2; fwr++){
251
252         multFWR = (fwr == 1 ? 2 : 1); //multFWR
253
254         for (fbr = 0; fbr <= 2; fbr++){
255
256             multFBR = (fbr == 1 ? 2 : 1); //multFBR
257
258
259             for (fixp = 0; fixp <= 1; fixp++){
260
261                 multEP = (fixp == 1 ? 14 : 1); //multEP
262
263                 //Multiplikatoren heranmultiplizieren
264                 res = multEP * multFWR * multFBR * count(fwr, fbr, fixp);
265                 printf("fwr = %d\tfbr = %d\tfixp = %d\tgerundet: %e\texakt: %1.0f\n", fwr, fbr, fixp, res, res);
266                 summe += res;
267             }
268         }
269     }
270
271     // Anzahl wird mit 2 multipliziert um die Schachpositionen in denen Schwarz am Zug ist zu berücksichtigen
272     printf("Totale Summe: %1.0f \n", 2 * summe);
273     return 1;
274 }
275

```

## meincode.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  extern double pow(double, double);
6
7  #define min(a,b) ((a) < (b) ? (a) : (b))
8  #define max(a,b) ((a) > (b) ? (a) : (b))
9
10 #define MNV 30           //Komponentenverschiebung im Feld mn
11 #define MBV 36           //Komponentenverschiebung im Feld mb
12
13
14 double fac[65];          // fac[n] = n!
15 double binom[65][11];    // binom[n][k] = Binomialkoeffizient von n über k
16 double choose2 [49][9][9]; // choose[n][j][k] = Multinomialkoeffizient  $n! / (j! * k! * (n - j - k)!)$ 
17 double mb [4][29][10][11]; // mb[kpos][s][wq][wr] = Obere Schranke aus Satz 5.3, wobei  $s + 36 = \text{space}$ 
18 double mn [34][11][11][11]; // mn[n][j][k][l] = Multinomialkoeffizient  $(n + 30)! / (j! * k! * l! * (n + 30 - j - k - l)!)$ 
19 double mopps_[9][33][9]; // Umsetzung von  $m(p,s,n)$  aus Satz 4.10
20 double pawns[2][9][9][9];
21
22 //Berechnet die Werte von fac[i]
23 void factorial() {
24     int i, j;
25     for (i=0; i<65; i++) {
26         fac[i] = 1.0;
27         for (j=2; j<=i; j++) {
28             fac[i] *= j;
29         }
30     }
31 }
32
33 // Berechnet die Werte von choose2[n][k1][k2]
34 void choose2function(){
35     int n, k1, k2;
36     for (n = 0; n <= 48; n++){
37         for (k1 = 0; k1 <= 8; k1++){
38             for (k2 = 0; k2 <= 8; k2++){
39                 if (n >= k1 + k2){
40                     choose2[n][k1][k2] = fac[n] / (fac[k1] * fac[k2] * fac[n - k1 - k2]);
41                 } else {
42                     choose2[n][k1][k2] = 0;
43                 }
44             }
45         }
46     }
47 }
48
49 //Berechnet die Werte von mopps[p][s][n]
50 double opps(int p, int s, int n) {
51     double res = 0.0;
52     if (p == 0 && s == 0) {
53         res = 1;
54     } else if (p == 0 || n == 0) {
55         res = 0;
56     } else {
57         res = mopps_[p][s][n - 1];
58         for (int i = 0; i <= (s < 4 ? s : 4); ++i) {
59             res += (5 - i) * mopps_[p - 1][s - i][n - 1];
60         }
61     }
62     return res;
63 }
64
65 //Weißt mopps[p][s][n], die in opps(p,s,n) berechneten Werte zu
66 void moppsfunction(){
67     double max = 0.0;
68     for (int p = 0; p <= 8; ++p) {
69         for (int s = 0; s <= 32; ++s) {
70             for (int n = 0; n <= 8; ++n) {
71                 mopps_[p][s][n] = opps(p, s, n);

```

```

72     }
73 }
74 }
75 }
76
77 // Berechnet die innere Summe in Satz 4.12
78 // dw = 1 <=> weißer Bauer in schwarzer fixierter Spalte
79 // db = 1 <=> schwarzer Bauer in weißer fixierter Spalte
80 // ds = ds1 + ds2 = Anzahl an eingeschlossener Felder in fixierten Spalten
81 double pawnsep (int dw, int db, int ds, int wp, int bp, int opp){
82     int opp_ = opp - dw - db;           // Mindestanzahl blockierter nicht-fixierter Spalten
83     int s;                               // Anzahl eingeschlossener Felder
84     double res = 0.0;
85     for (s = 0; s <= 4 * opp_; s++){
86         if(bp + dw >= opp && wp + db >= opp){ // Damit keine negative Komponente von choose2 aufgerufen wird
87             res += mopps_[opp_][s][6] * choose2[44 - opp - opp_ - ds - s][wp + db - opp][bp + dw - opp];
88         }
89     }
90     return res;
91 }
92
93
94 void pawnsfunction(){
95     int s, wp, bp, opp;
96     int ds1, ds2;
97     double res;
98
99     // Fall: Kein Enpassant -> fixp = 0
100    // Berechnet obere Schranke aus Satz 4.11
101    for (wp = 0; wp <= 8; wp++){
102        for (bp = 0; bp <= 8; bp++){
103            for (opp = 0; opp <= 8; opp++){
104                if (opp > min(wp, bp)){ // Es darf nicht mehr blockierte Spalten als Bauern einer Seite geben
105                    pawns[0][wp][bp][opp] = 0;
106                } else {
107                    res = 0.0;
108                    for (s = 0; s <= 4 * opp; s++){
109                        res += mopps_[opp][s][8] * choose2[48 - 2 * opp - s][wp - opp][bp - opp];
110                    }
111                    pawns[0][wp][bp][opp] = res;
112                }
113            }
114        }
115    }
116
117    // Fall: Enpassant -> fixp = 1
118    // Berechnet obere Schranke aus Satz 4.12
119    for (wp = 0; wp <= 8; wp++){
120        for (bp = 0; bp <= 8; bp++){
121            for (opp = 0; opp <= 8; opp++){
122                res = 0;
123                if (opp > min(wp + 1, bp + 1)){ // Es darf nicht mehr blockierte Spalten als Bauern einer Seite geben
124                    pawns[1][wp][bp][opp] = 0;
125                } else {
126                    res = pawnsep(0, 0, 0, wp, bp, opp);
127                    for (ds1 = 0; ds1 <= 2 && opp > 1; ds1++){
128                        for (ds2 = 0; ds2 <= 1; ds2++){
129                            res += pawnsep(1, 1, ds1 + ds2, wp, bp, opp);
130                        }
131                    }
132                    for (ds1 = 0; ds1 <= 2 && opp > 0; ds1++){
133                        res += pawnsep(1, 0, ds1, wp, bp, opp);
134                    }
135                    for (ds2 = 0; ds2 <= 1 && opp > 0; ds2++){
136                        res += pawnsep(0, 1, ds2, wp, bp, opp);
137                    }
138                    pawns[1][wp][bp][opp] = res;
139                }
140            }
141        }
142    }
143 }
144 }
145
146 //Beginn neuer Funktionen
147

```

```

148 // Berechnet n über k und speichert es in binom[n][k]
149 void binomial(){
150     int i, j;
151     for (i=0; i<65; i++) {
152         for (j=0; j<11; j++) {
153             if (i >= j) {
154                 binom[i][j] = fac[i] / (fac[j] * fac[i-j]);
155             } else {
156                 binom[i][j] = 0;
157             }
158         }
159     }
160 }
161
162 //Speichert den Wert  $(n + 30)! / (j! * k! * l! * (n + 30 - j - k - l)!)$  in mn[n][j][k][l]
163 void multinom(){
164     int spaces, f1, f2, f3; // Spaces wird hier um MNV verschoben
165     for (spaces = MNV; spaces <= 63; spaces++){
166         for (f1 = 0; f1 <= 10; f1++){
167             for (f2 = 0; f2 <= 10; f2++){
168                 for (f3 = 0; f3 <= 10; f3++){
169                     mn[spaces-MNV][f1][f2][f3] = fac[spaces] / (fac[f1] * fac[f2] * fac[f3] * fac[spaces - f1 - f2 - f3]);
170                 }
171             }
172         }
173     }
174 }
175 }
176
177 // Berechnet die obere Schranke aus Lemma 5.3 und speichert sie im Array mb
178 void multiplebinomrook (){
179     int kpos, space, wq, wr;
180     int qspaces;
181     for (kpos = 0; kpos <= 3; kpos++){
182         switch (kpos) {
183             case 0: qspaces = 1; break; // Ecke
184             case 1: qspaces = 2; break; // Letzte Reihe
185             case 2: qspaces = 0; break; // Bauernterritorium
186             case 3: qspaces = 2; break; // Fixierter König
187         }
188         for (space = MBV; space <= 64; space++){
189             for (wq = 0; wq <= 9; wq++){
190                 for (wr = 0; wr <= 10; wr++){
191                     mb[kpos][space-MBV][wq][wr] = binom[space - qspaces - 1][wq] * binom[space - wq - qspaces - 1][wr];
192                 }
193             }
194         }
195     }
196 }
197 // Ende neuer Funktionen
198
199 // Die count-Funktion wurde modifiziert, hier werden zur Übersichtlichkeit nur die modifizieren Stellen kommentiert
200
201 double count (int fwr, int fbr, int fixp){
202     double summe = 0.0;
203     double h = 0.0; // Hilfsvariable, damit der Code nicht zu weit nach rechts geht und gedruckt werden kann
204     int np = 8 - fixp;
205     int fixwk = (fwr != 0 ? 1 : 0);
206     int uwr, wk, wq, wnp1, wr, wb, wnp2, wnp3, wn, wnp4, wproms, wp, wpx;
207     int ubr, bk, bq, bnp1, br, bb, bnp2, bnp3, bn, bnp4, bproms, bp, bpx;
208     int fixbk = (fbr != 0 ? 1 : 0);
209     int wpcs, bpcs, caps, maxuwp, maxubp, minopp, space;
210     // wprod und bprod werden nicht mehr benötigt, da die Multinomialkoeffizienten mit dem mn-Array berechnet werden
211
212     // kpos und kmult kennen wir aus Kapitel 5, multFWK gibt die Anzahl an Möglichkeiten an den weißen König zu platzieren
213     // Die Platzierung des weißen Königs ist nach den Bauern, dem schwarzen König, den weißen Damen und weißen nicht-fixierten Türmen
214     int kpos, multFWK, kmult;
215     uwr = 2 - fwr;
216     wk = floor(uwr / 2);
217
218     // Hier wird geschaut, ob es keinen fixierten König gibt.
219     if (fbr == 0){
220
221         // Hier werden alle möglichen kpos eines nicht-fixierten Königs durchgegangen
222         for (kpos = 0; kpos <= 2; kpos++){
223             for (wq = 0; wq <= 1 + np; wq++){

```

```

224 wnp1 = np - max(wq - 1, 0);
225 for (wr = 0; wr <= uwr + wnp1; wr++){
226     wnp2 = wnp1 - max(wr - uwr, 0);
227     for (wb = 0; wb <= 2 + wnp2; wb++){
228         wnp3 = wnp2 - max(wb - 2, 0);
229         for (wn = 0; wn <= 2 + wnp3; wn++){
230             wnp4 = wnp3 - max(wn - 2, 0);
231             wproms = np - wnp4;
232             for (wp = 0; wp <= wnp4; wp++){
233                 wpx = np - wp - wproms;
234                 wpcs = wk + wq + wr + wb + wn;
235                 ubr = 2 - fbr;
236                 bk = floor(ubr / 2);
237                 for (bq = 0; bq <= 1 + np; bq++){
238                     bnp1 = np - max(bq - 1, 0);
239                     for (br = 0; br <= ubr + bnp1; br++){
240                         bnp2 = bnp1 - max(br - ubr, 0);
241                         for (bb = 0; bb <= 2 + bnp2; bb++){
242                             bnp3 = bnp2 - max(bb - 2, 0);
243                             for (bn = 0; bn <= 2 + bnp3; bn++){
244                                 bnp4 = bnp3 - max(bn - 2, 0);
245                                 bproms = np - bnp4;
246                                 for (bp = 0; bp <= bnp4; bp++){
247
248                                     bpx = np - bp - bproms;
249                                     bpcs = bk + bq + br + bb + bn;
250                                     caps = 32 - 2 * fixp - fixwk - fixbk - fwr - fbr - wp - bp - wpcs - bpcs;
251                                     maxuwp = bpx + caps - wproms;
252                                     maxubp = wpx + caps - bproms;
253                                     if (maxuwp < 0 || maxubp < 0){
254                                         continue;
255                                     } else {
256                                         minopp = max(0, fixp + wp - maxuwp);
257                                         // Wenn der weiße König fixiert ist, gibt es eine Möglichkeit den weißen König
258                                         // zu platzieren, ansonsten 63 - wr - wq - 4 * fixp - fbr - wp - bp
259                                         space = 64 - 4 * fixp - fwr - fbr - wp - bp - fixwk;
260                                         multFWK = (fwr > 0 ? 1 : space - 1 - wr - wq);
261                                         h = 0;
262                                         switch (kpos){
263                                             case 0: kmult = 4 - fwr; break;
264                                             case 1: kmult = 12 - 3 * fixwk - fwr; break;
265                                             case 2: kmult = 48 - 4 * fixp - wp - bp - 3 * fixwk - fwr; break;
266                                         }
267                                         // Der Multinomialkoeffizient von Tromp wird mithilfe des Ergebnisses der oberen
268                                         // Schranke aus Satz 5.3, multFWK und der Verteilung der restlichen Figuren
269                                         // als Multinomialkoeffizient ersetzt
270                                         h = pawns[fixp][wp][bp][minopp] * mb[kpos][space - MBV][wq][wr] * multFWK;
271                                         h *= kmult * mn[space - 2 - wq - wr - MNV + fixwk][wn][wb][bq];
272                                         h *= mn[space - 2 - wq - wn - wr - wb - bq - MNV + fixwk][br][bb][bn];
273                                         summe += h;
274                                     }
275                                 }
276                             }
277                         }
278                     }
279                 }
280             }
281         }
282     }
283 }
284
285 } else {
286     // Wenn der König fixiert ist, ist in jedem Fall kpos = 3 gegeben und damit auch kmult = 1
287     kpos = 3;
288     kmult = 1;
289     for (wq = 0; wq <= 1 + np; wq++){
290         wnp1 = np - max(wq - 1, 0);
291         for (wr = 0; wr <= uwr + wnp1; wr++){
292             wnp2 = wnp1 - max(wr - uwr, 0);
293             for (wb = 0; wb <= 2 + wnp2; wb++){
294                 wnp3 = wnp2 - max(wb - 2, 0);
295                 for (wn = 0; wn <= 2 + wnp3; wn++){
296                     wnp4 = wnp3 - max(wn - 2, 0);
297                     wproms = np - wnp4;
298                     for (wp = 0; wp <= wnp4; wp++){

```



```

300     wpx = np - wp - wproms;
301     wpcs = wk + wq + wr + wb + wn;
302
303     ubr = 2 - fbr;
304     bk = floor(ubr / 2);
305     np = 8 - fixp;
306     for (bq = 0; bq <= 1 + np; bq++){
307         bnp1 = np - max(bq - 1, 0);
308         for (br = 0; br <= ubr + bnp1; br++){
309             bnp2 = bnp1 - max(br - ubr, 0);
310             for (bb = 0; bb <= 2 + bnp2; bb++){
311                 bnp3 = bnp2 - max(bb - 2, 0);
312                 for (bn = 0; bn <= 2 + bnp3; bn++){
313                     bnp4 = bnp3 - max(bn - 2, 0);
314                     bproms = np - bnp4;
315                     for (bp = 0; bp <= bnp4; bp++){
316
317                         bpx = np - bp - bproms;
318                         bpcs = bk + bq + br + bb + bn;
319                         caps = 32 - 2 * fixp - fixwk - fixbk - fwr - fbr - wp - bp - wpcs - bpcs;
320                         maxuwp = bpx + caps - wproms;
321                         maxubp = wpx + caps - bproms;
322                         if (maxuwp < 0 || maxubp < 0){
323                             continue;
324                         } else {
325                             minopp = max(0, fixp + wp - maxuwp);
326                             space = 64 - 4 * fixp - fwr - fbr - wp - bp - fixwk;
327                             // Wenn der weiße König fixiert ist, gibt es eine Möglichkeit den weißen König zu
328                             // platzieren, ansonsten 63 - wr - wq - 4 * fixp - fbr - wp - bp
329                             multFWK = (fwr > 0 ? 1 : space - 1 - wr - wq);
330                             h = 0;
331
332                             // Der Multinomialkoeffizient von Tromp wird mithilfe des Ergebnisses der oberen
333                             // Schranke aus Satz 5.3, multFWK und der Verteilung der restlichen Figuren als
334                             // Multinomialkoeffizient ersetzt
335                             h = pawns[fixp][wp][bp][minopp] * mb[kpos][space - MBV][wq][wr] * multFWK;
336                             h *= kmult * mn[space - 2 - wq - wr - MNV + fixwk][wn][wb][bq];
337                             h *= mn[space - 2 - wq - wn - wr - wb - bq - MNV + fixwk][br][bb][bn];
338                             summe += h;
339                         }
340                     }
341                 }
342             }
343         }
344     }
345 }
346
347 }
348
349 }
350 }
351 return summe;
352 }
353
354 // In main gibt es wenig Unterschiede
355 // Es werden lediglich mehr Arrays initialisiert und am Ende wird noch ein Vergleich zu Tromps Code gezogen
356 int main () {
357     //Initialisierung der Arrays
358     factorial();
359     binomial();
360     choose2function();
361     multinom();
362     multiplebinomrook();
363     moppsfunction();
364     pawnsfunction();
365     int fwr, fbr, fixp;
366     double res = 0.0;
367     double summe = 0.0;
368     double multFWR, multEP, multFBR;
369
370     // Alle möglichen Werte für fwr, fbr und fixp durchlaufen
371     for (fwr = 0; fwr <= 2; fwr++){
372         multFWR = (fwr == 1 ? 2 : 1); //multFWR
373
374
375

```

```

376     for (fbr = 0; fbr <= 2; fbr++){
377
378         multFBR = (fbr == 1 ? 2 : 1); //multFBR
379
380
381         for (fixp = 0; fixp <= 1; fixp++){
382
383             multEP = (fixp == 1 ? 14 : 1); //multEP
384
385             //Multiplikatoren heranmultiplizieren
386             res = multEP * multFWR * multFBR * count(fwr, fbr, fixp);
387             printf("fwr = %d\tfbr = %d\tfixp = %d\tgerundet: %e\texakt: %1.0f\n", fwr, fbr, fixp, res, res);
388             summe += res;
389         }
390     }
391 }
392
393 // Ergebnis von Tromp
394 double haskellergebnis = 8726713169886222032347729969256422370854716254.0;
395
396 printf("Totale Summe: %1.0f \n", 2 * summe);
397 printf("Verbesserung um %1.1f Prozent\n", 100 * ((haskellergebnis - 2 * summe)/haskellergebnis));
398 return 1;
399 }

```